



ChatGPT



Gemini



Qwen

(Vision) Transformers

draft version, do not print

Computer Vision and Artificial
Intelligence

Dr Varun Ojha

varun.ojha@ncl.ac.uk

ojhavk.github.io

School of Computing
Newcastle University



Deepseek R1



Sora



midjourney



Mistral AI



runway

Google
Veo 3

Learning objectives (Vision Transformer)

By the end of this week, you will be able to:

- Learn the concepts of Transformer Models
- Understand the the basic building block of Transformers
- Understand an image classification using Vision Transformers
- Know the limitations and advantages of Transformers

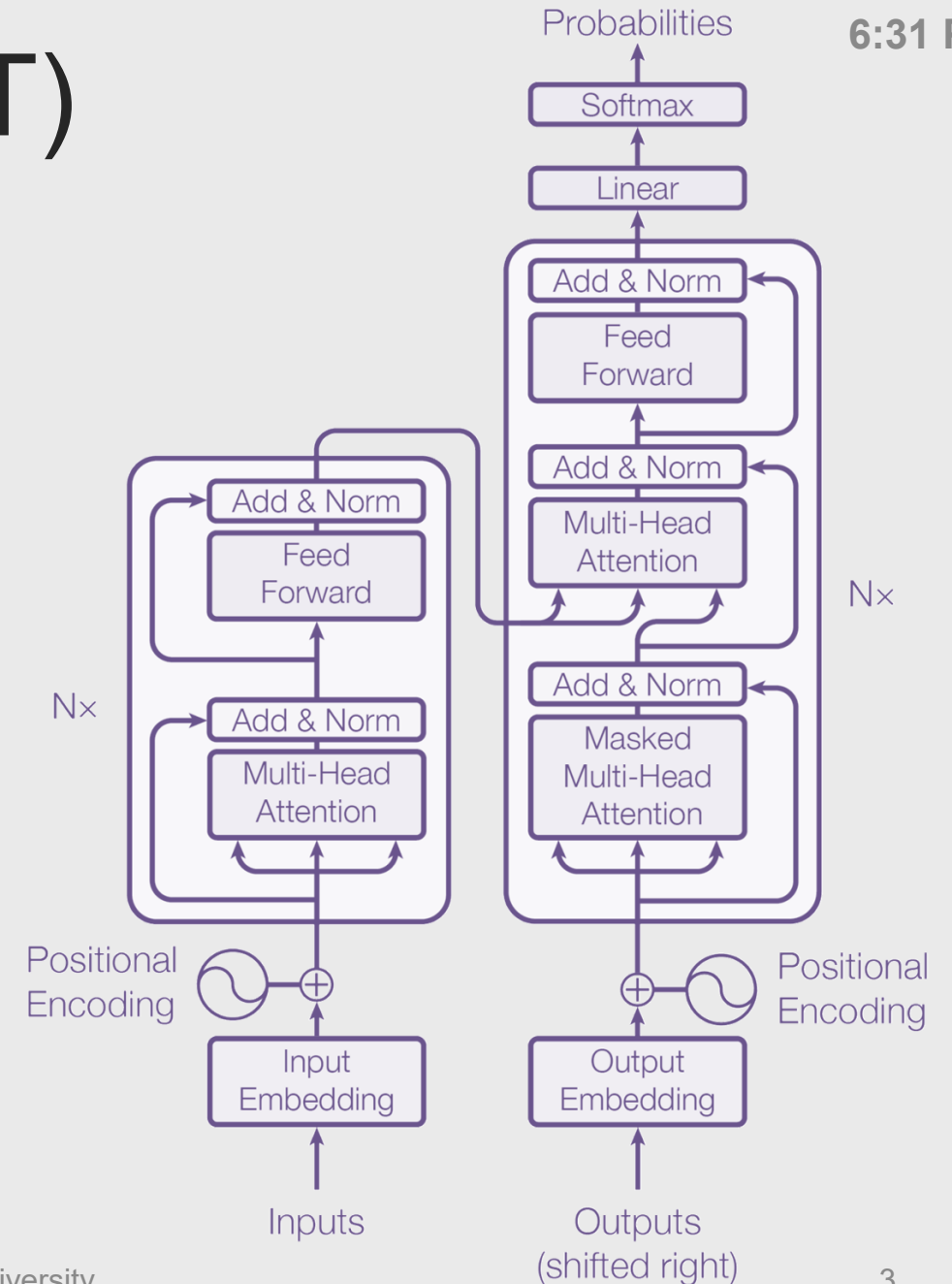
Content of this week (ViT)

- **Part 1: Self-Attention Block**

- Basic concept of Transformers
- Word Embedding
- Self-Attention Mechanism

- **Part 2: Vision Transformer**

- Basic concept of Vision Transformers
- Architecture of a Vision Transformer
- Performance of Vision Transformer



Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Advance AI Models

6:31 PM



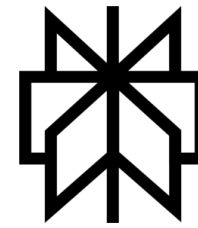
ChatGPT



Gemini



Qwen



perplexity



Mistral AI



Deepseek R1



Sora



midjourney



runway

Google
Vevo 3

Unstructured Data

Suppose this text is a message in your mailbox

History of natural language processing : The history of natural language processing (NLP) generally started in the 1950s, although work can be found from earlier periods. In 1950, Alan Turing published an article titled "Computing Machinery and Intelligence" which proposed what is now called the Turing test as a criterion of intelligence[clarification needed].

The Georgetown experiment in 1954 involved fully automatic translation of more than sixty Russian sentences into English. The authors claimed that within three or five years, machine translation would be a solved problem.[2] However, real progress was much slower, and after the ALPAC report in 1966, which found that ten-year-long research had failed to fulfill the expectations, funding for machine translation was dramatically reduced. Little further research in machine translation was conducted until the late 1980s when the first statistical machine translation systems were developed.

Some notably successful natural language processing systems developed in the 1960s were SHRDLU, a natural language system working in restricted "blocks worlds" with restricted vocabularies, and ELIZA, a simulation of a Rogerian psychotherapist, written by Joseph Weizenbaum between 1964 and 1966. Using almost no information about human thought or emotion, ELIZA sometimes provided a startlingly human-like interaction. When the "patient" exceeded the very small knowledge base, ELIZA might provide a generic response, for example, responding to "My head hurts" with "Why do you say your head hurts?".Text

Source: https://en.wikipedia.org/wiki/Natural_language_processing

Dataset

No apparent structure in the data

Is it possible to classify a SPAM or Genuine message

Is it possible to identify a positive or negative sentence?

Is it possible to predict the following email text or a reply from the current?

Is it possible to translate?

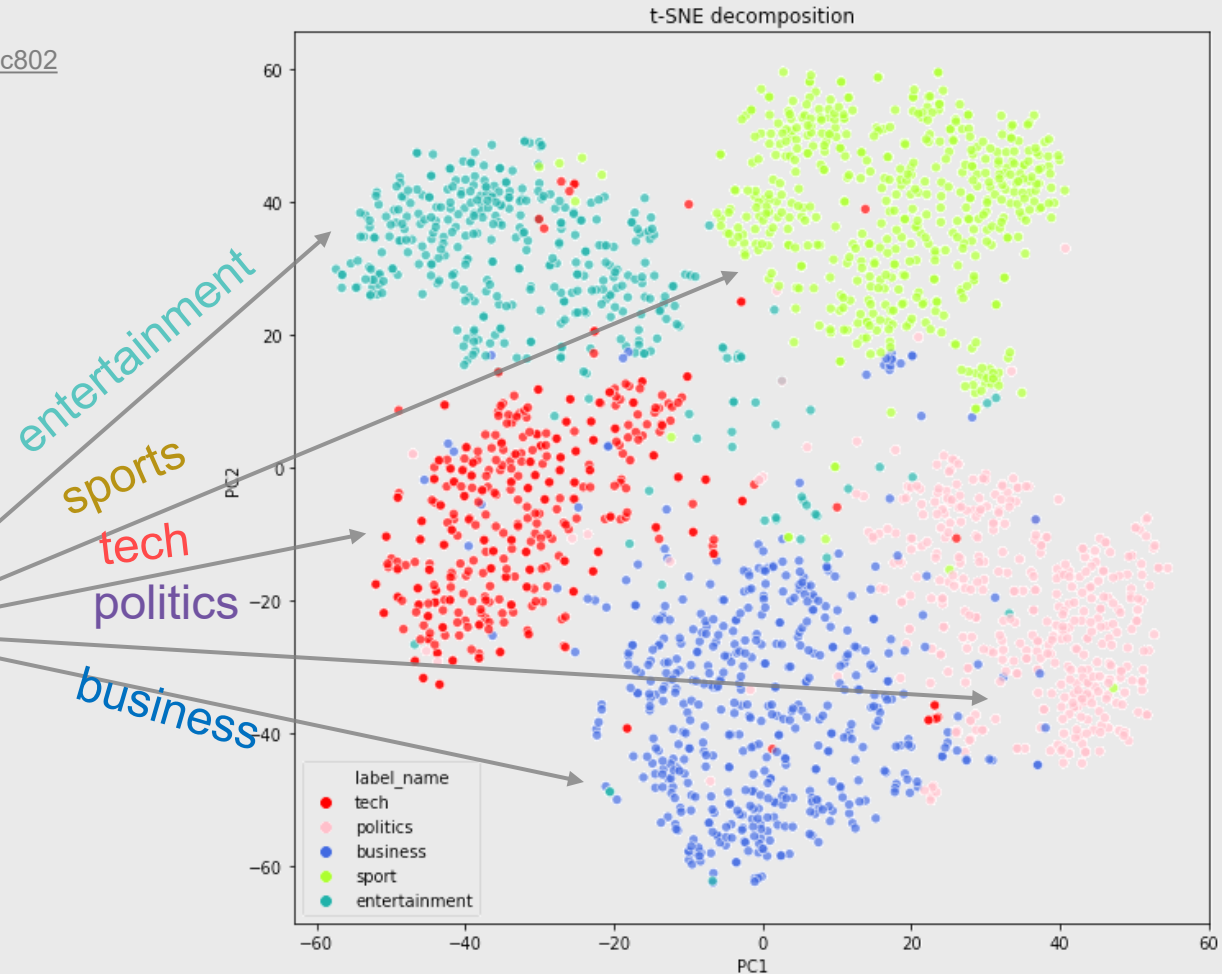
Text Classification

Example Source:

<https://towardsdatascience.com/text-classification-in-python-dd95d264c802>

Question: The following text belongs to which category?

The use of electronic devices in the Commons chamber has long been frowned on. The sound of a mobile phone or a pager can result in a strong rebuke from either the Speaker or his deputies. The Speaker chairs debates in the Commons and is charged with ensuring order in the chamber and enforcing rules and conventions of the House. He or she is always an MP chosen by colleagues who, once nominated, gives up all party-political allegiances.



Word Embedding

Given a word W (e.g. “intelligence”) we want W to be a real vector of dimension d . Dimension d is also called word embedding dimension.

$$W: \text{words} \rightarrow \mathbb{R}^d$$

 Real values of dimension d

$$\text{“intelligence”} \rightarrow (w_1, w_2, \dots, w_d) \rightarrow (0.1, -0.8, \dots, 0.9)$$

Word Embedding

$S_1 = \text{"I love artificial intelligence"}$

$S_2 = \text{"I like computational intelligence"}$

We create a vocabulary V collecting all unique words.

$V = \{\text{"I", "love", "like", "artificial", "computational", "intelligence"}\}$

 This is a set

For this example, vocabulary size $|V| = 6$

 Cardinality of the set

We create a vocabulary V
collecting all unique words.

$V = \{\text{"I", "love", "like",
"artificial", "computational",
"intelligence"}\}$

For this example,
vocabulary size $|V| = 6$

The current online edition of **Oxford English Dictionary** contains more than 600,000 words



SAMUEL JOHNSON, LL.D.

*Engraved by The Lock from an Original Painting by Sir Joshua Reynolds
in the possession of R. Langton Esq.*

Published in the Strand, March 10th 1773, by J. Langman, Stationer, &c.

A
DICTIONARY
OF THE
ENGLISH LANGUAGE:
IN WHICH
THE WORDS ARE DEDUCED FROM THEIR ORIGINALS,
AND
ILLUSTRATED IN THEIR DIFFERENT SIGNIFICATIONS
BY
EXAMPLES from the best WRITERS,
TO WHICH ARE PREFIXED,
A HISTORY OF THE LANGUAGE,
AND
AN ENGLISH GRAMMAR.

By SAMUEL JOHNSON, LL.D.

THE SEVENTH EDITION.

*Com tabula animarum eruditiorum sunt benefici:
Audent quocumque positis splendore iudicant,
Et sine pondere erant, et hunc indignum iudicant,
Verba moris laus, quoniam iustis reddunt,
Et rectoribus inter potentia Velle:
Obsecro dia populo bonus erant, atque
Probat in locum ferenda societas rerum,
Que prius memoris Catoles atque Catoles
Nunc ista infamia prout et deferta vetulas. — Hor.*

L O N D O N:

Printed for J. F. and C. RIVINGTON, L. DAVIS, T. PAYNE and SON, W. OWEN, T. LINGMAN, B. LAW, J. DODDLEY, C. DILLY,
W. LOWNDSE, G. G. J. and J. ROBINSON, T. CADELL, JR. JOHNSON, J. ROBINSON, W. RICHARDSON, J. NICHOLS,
R. BALDWIN, W. GOLDSMITH, J. MURRAY, W. STEWART, P. ELMLEY, W. FOX, S. HAYES,
W. BENT, T. and J. EGERTON, D. OGLEBY, and E. NEWBURY.

MDCCCLXXXV.

Word Embedding: Word $\rightarrow$ Integer

$V = \{\text{"I"}, \text{"love"}, \text{"artificial"}, \text{"computational"}, \text{"intelligence"}, \text{"like"}\}$

I $\rightarrow$ 0

love $\rightarrow$ 1

like $\rightarrow$ 2

artificial $\rightarrow$ 3

computational $\rightarrow$ 4

intelligence $\rightarrow$ 5

Word Embedding: Integer $\rightarrow$ Word

$V = \{\text{"I"}, \text{"love"}, \text{"artificial"}, \text{"computational"}, \text{"intelligence"}, \text{"like"}\}$

$0 \rightarrow \text{I}$

$1 \rightarrow \text{love}$

$2 \rightarrow \text{like}$

$3 \rightarrow \text{artificial}$

$4 \rightarrow \text{computational}$

$5 \rightarrow \text{intelligence}$

One-Hot Encoding

$V = \{\text{"I", "love", "like", "artificial", "computational", "intelligence"}\}$

$$\text{"I"} = \begin{bmatrix} \mathbf{1} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{"love"} = \begin{bmatrix} 0 \\ \mathbf{1} \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

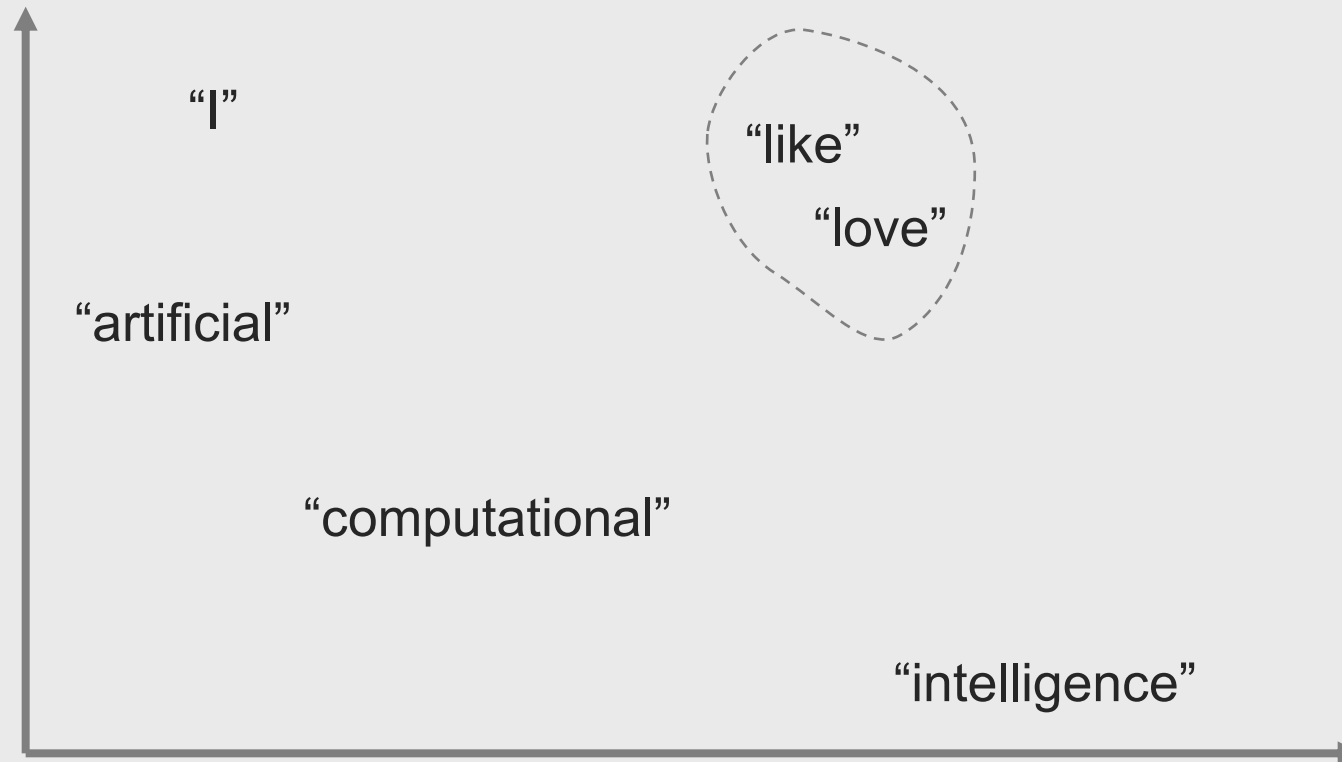
$$\text{"like"} = \begin{bmatrix} 0 \\ 0 \\ \mathbf{1} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{"artificial"} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \mathbf{1} \\ 0 \\ 0 \end{bmatrix}$$

$$\text{"computational"} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{1} \\ 0 \end{bmatrix}$$

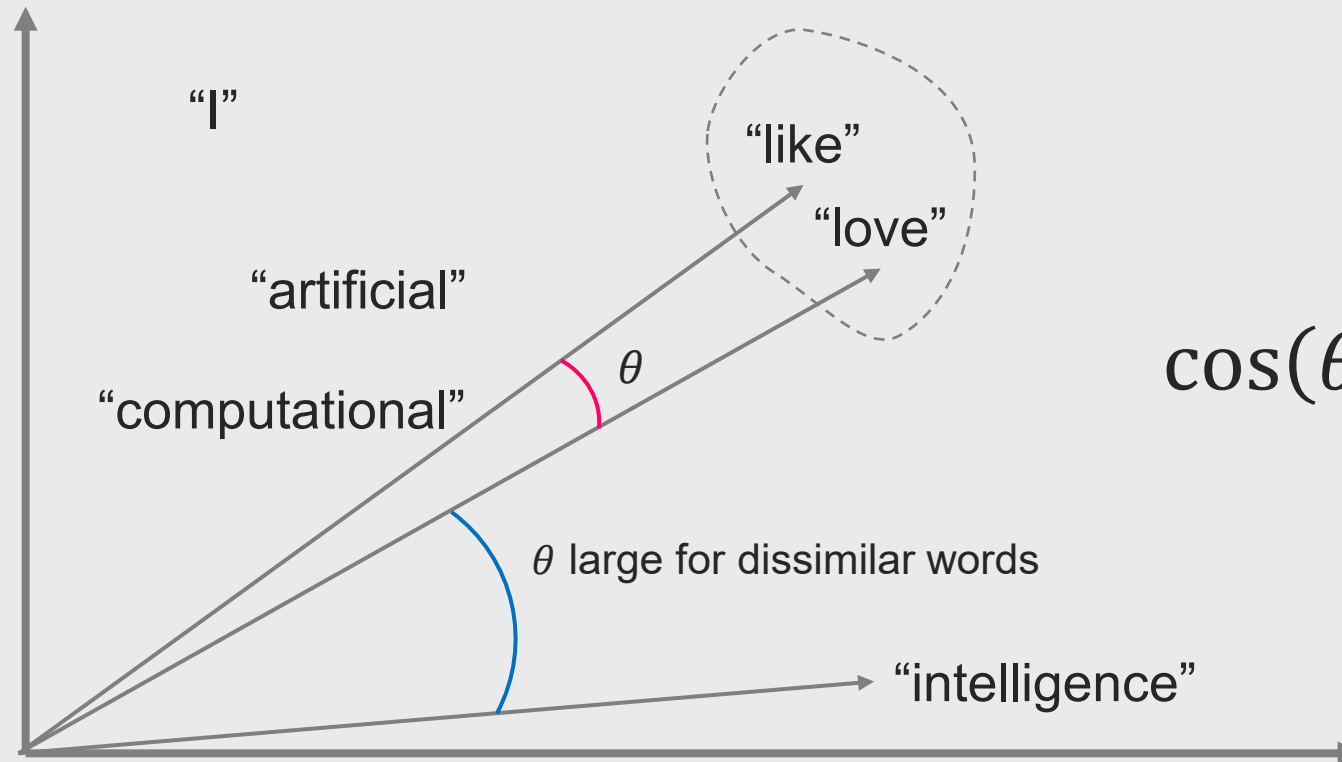
$$\text{"intelligence"} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{1} \end{bmatrix}$$

Similarity between words?



Objective is to place similar words close to each other

Similarity between words?

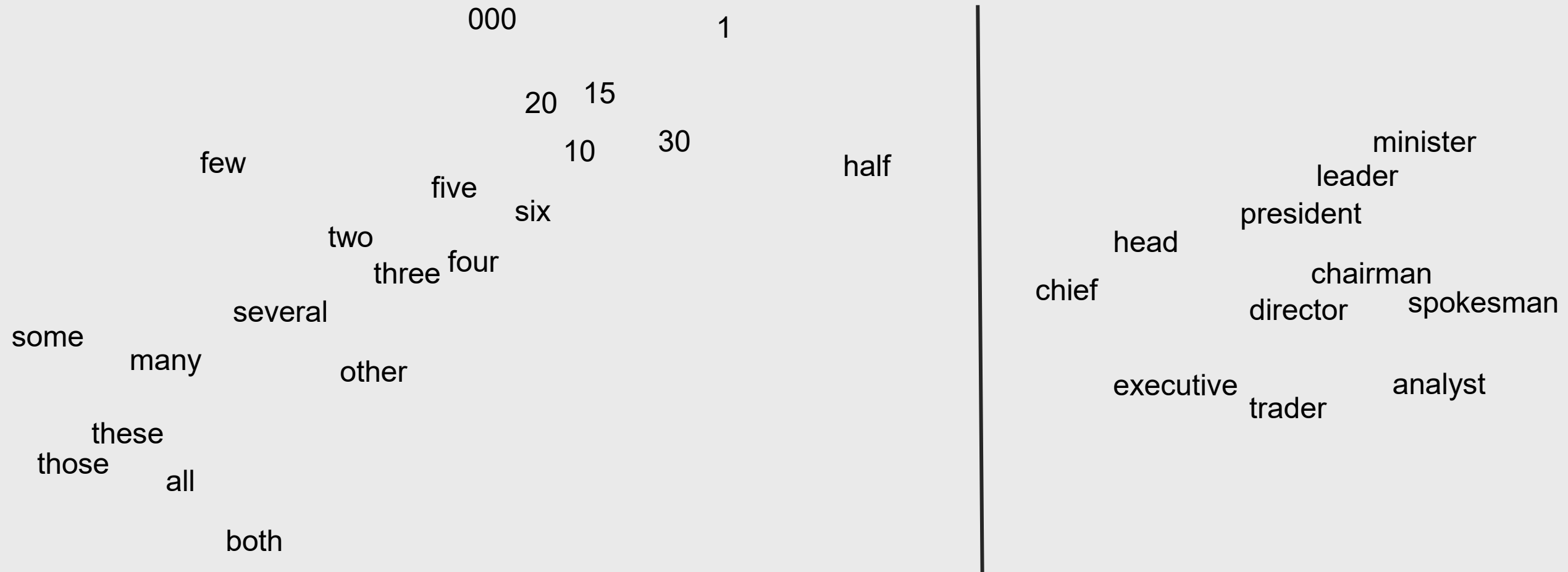


$$\cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Objective is to place similar words close to each other

t-SNE visualisation of words

Turian *et al.* (2010)

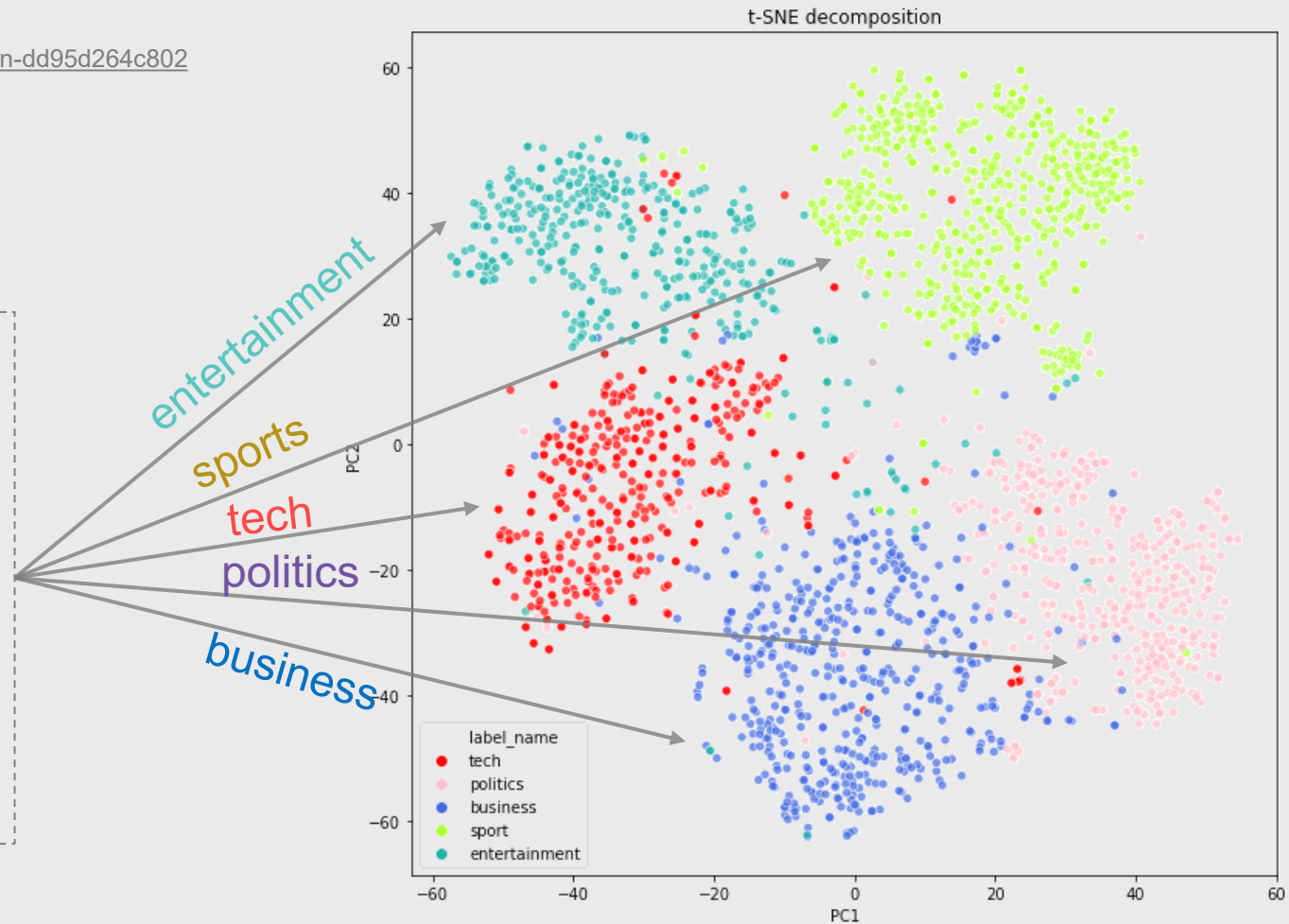


t-SNE visualisation

Example Source:

<https://towardsdatascience.com/text-classification-in-python-dd95d264c802>

All similar topics
are closer to
each other



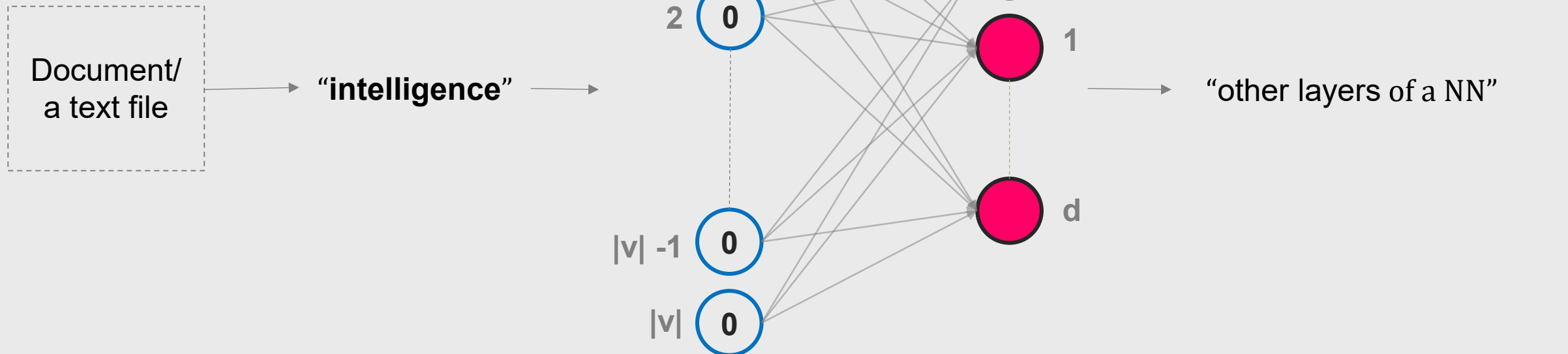
Word Embedding: Objective

Given a word W (e.g. “intelligence”) we want to W a real vector of dimension d

$$\mathbf{W}: \text{words} \rightarrow \mathbb{R}^d$$

$$\text{“intelligence”} \rightarrow (w_1, w_2, \dots, w_d) \rightarrow (0.1, -0.8, \dots, 0.9)$$

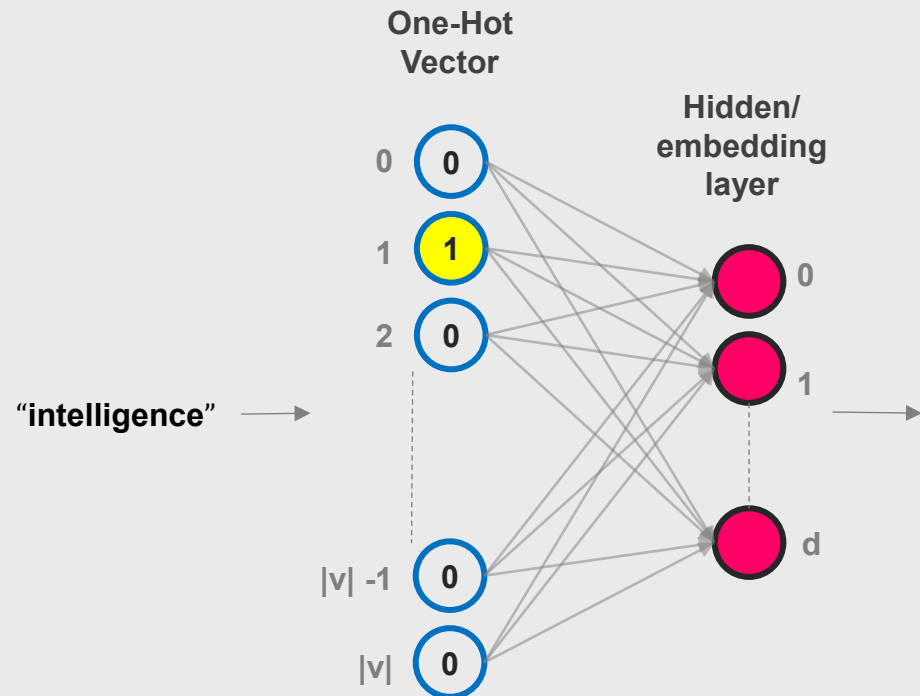
Word Embedding



Check online here: <https://ronxin.github.io/wavi/>

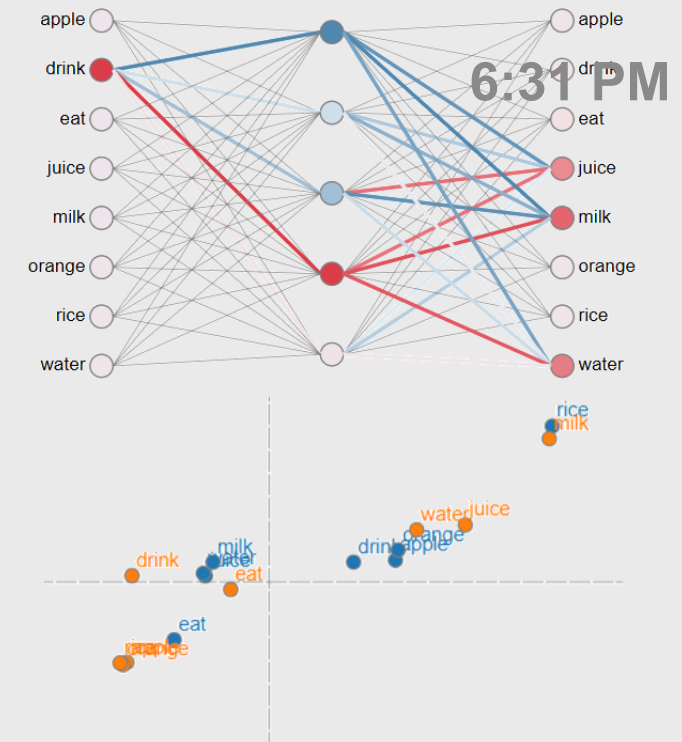
Word Embedding

Check online here: <https://ronxin.github.io/wevi/>



$$\text{“intelligence”} = [0 \quad 1 \quad 0 \quad \dots \quad 0 \quad 0] \times \begin{bmatrix} 0.5 & 4.6 & \dots & 0.9 \\ 0.1 & -0.8 & \dots & 0.7 \\ \vdots & \vdots & & \vdots \\ 0.6 & 0.8 & \dots & 0.3 \\ 0.3 & -0.6 & \dots & -0.8 \\ -0.5 & 0.5 & \dots & 0.1 \end{bmatrix}$$

$$(1 \times |V|) \cdot (|V| \times d) \Rightarrow (1 \times d)$$

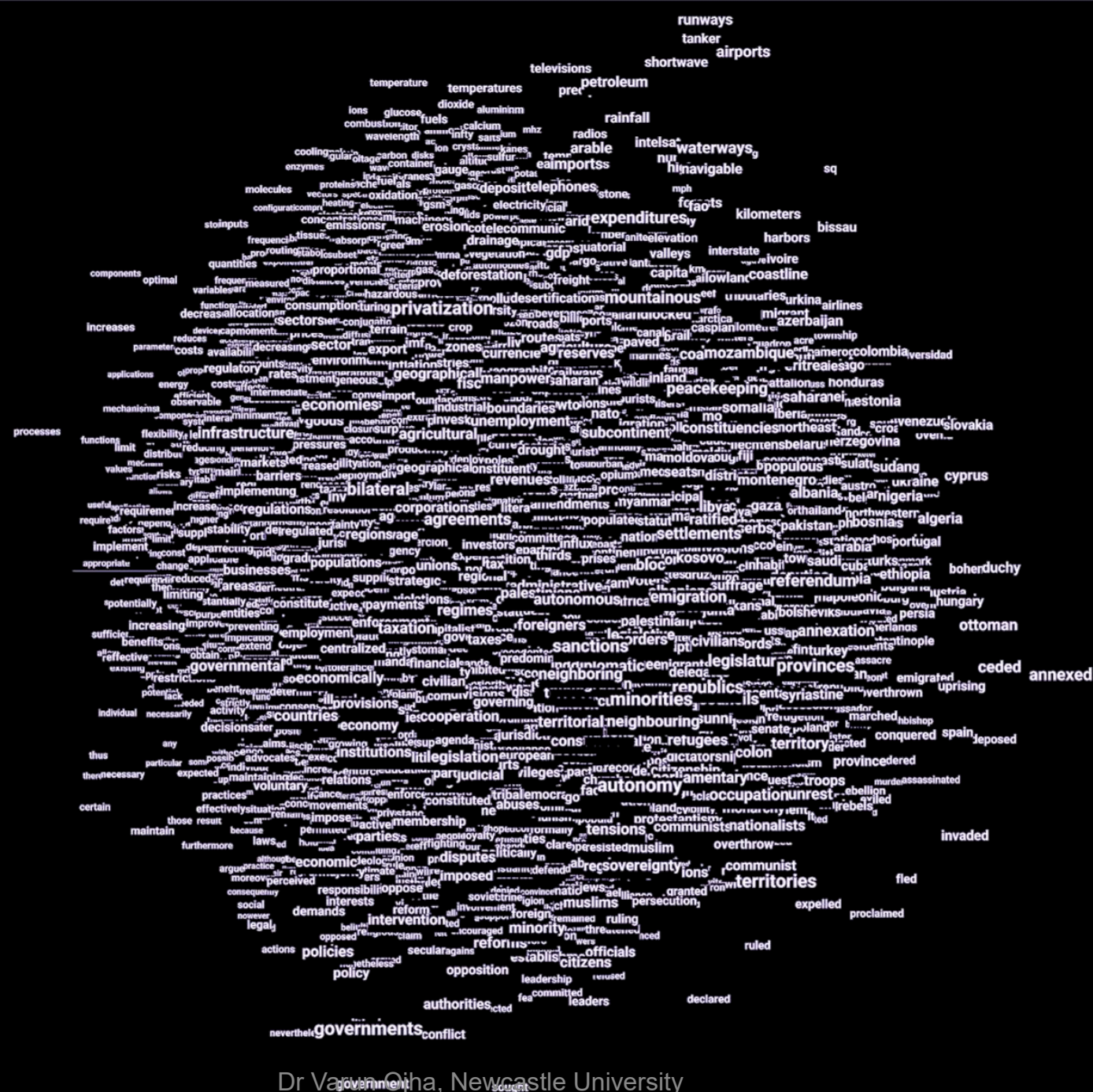


Word Embedding

$$\begin{aligned}
 \text{"intelligence"} &= [0 \quad 1 \quad 0 \quad \dots \quad 0 \quad 0] \times \begin{bmatrix} 0.5 & 4.6 & \dots & 0.7 \\ 0.1 & -0.8 & \dots & 0.9 \\ 0.6 & 0.8 & \dots & 0.3 \\ 0.3 & -0.6 & \dots & -0.8 \\ -0.5 & 0.5 & \dots & 0.1 \end{bmatrix} \\
 &= [0.1, -0.8, \dots, 0.9]
 \end{aligned}$$

Word Embedding Word2Vec 10K

6:31 PM



How to processes text data (a sentence or paragraph or a whole book)

S_1 = “I love artificial intelligence”

S_2 = “I like computational intelligence”

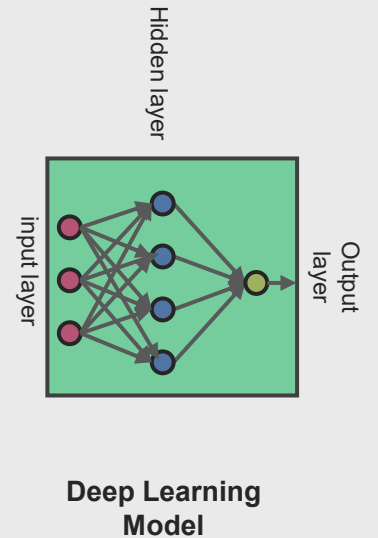
For this two sentences we have a vocabulary V :

$V = \{\text{“I”, “love”, “like”, “artificial”, “computational”, “intelligence”}\}$

The vocabulary size $|V| = 6$

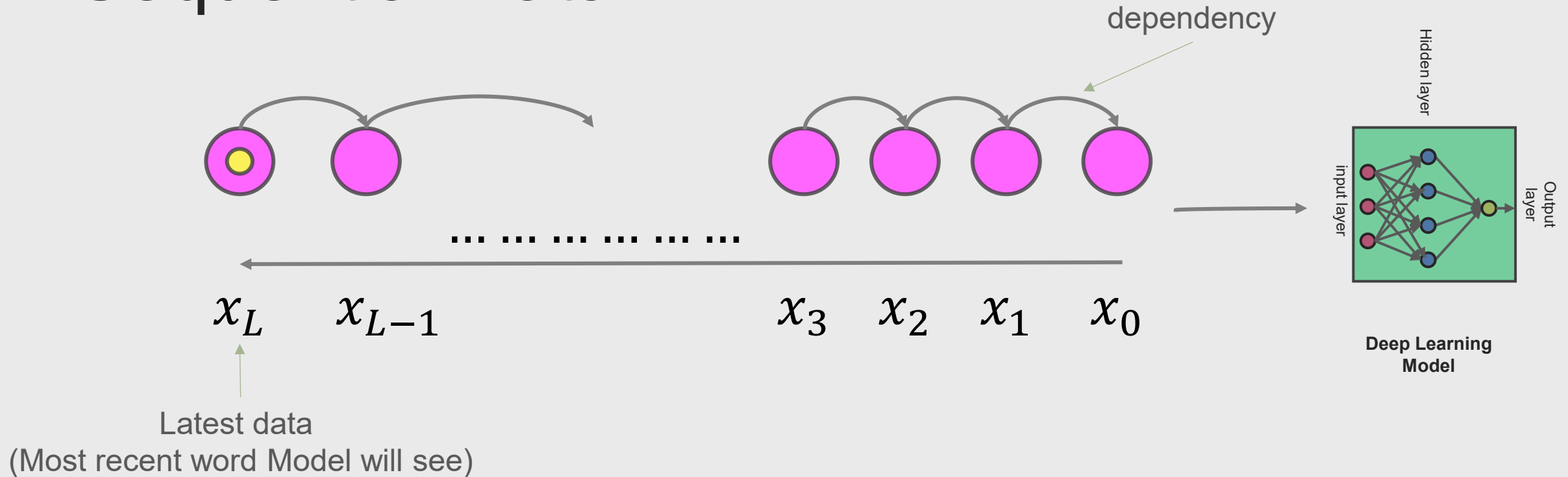
Text (a sentence) is a Sequential Data

How data will be fed to our deep learning model



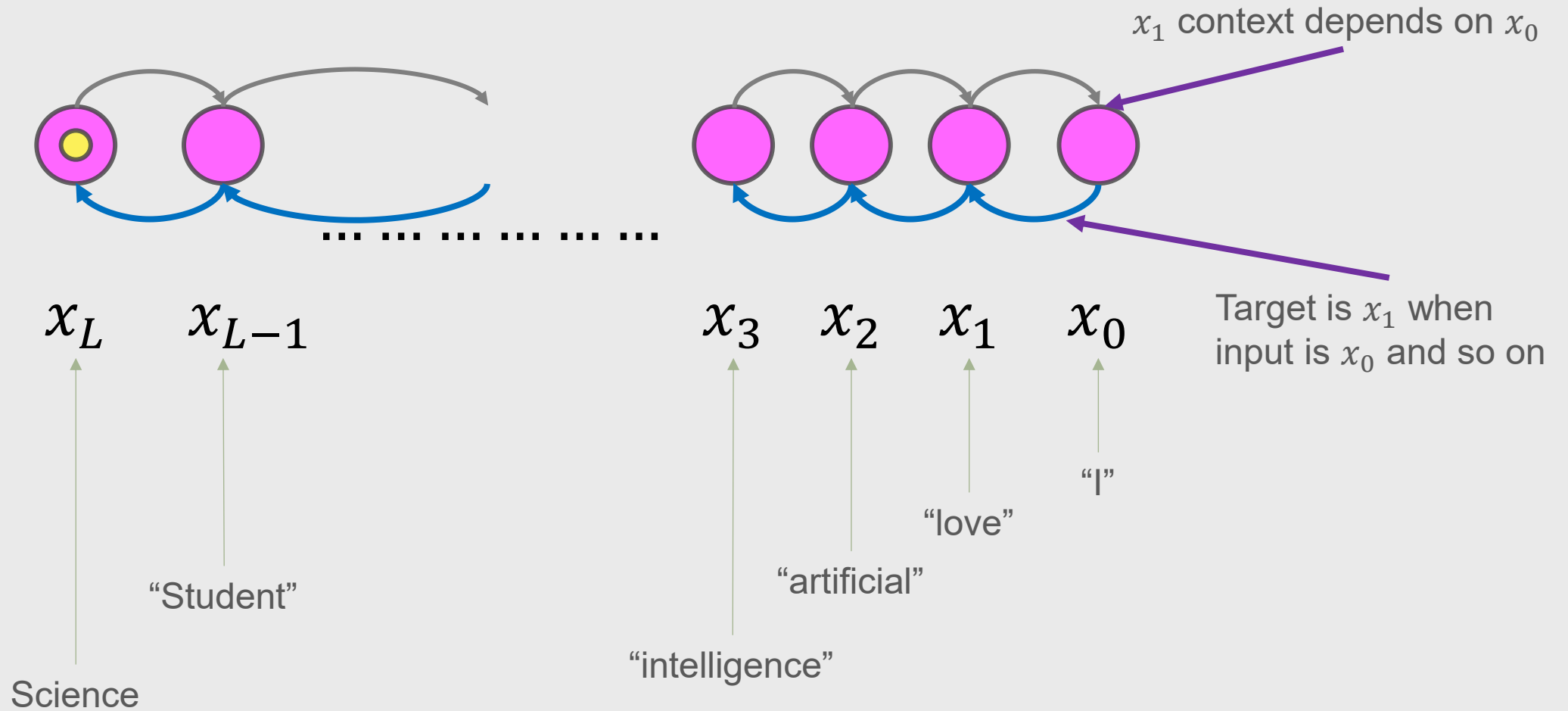
Let's say, we have data (document) $\mathbf{x} = \{x_0, x_1, x_2, \dots, x_L\}$ of length L arriving at time $t = 0$ until time $t = L$

Sequential Data

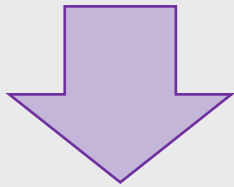


data set $\mathbf{x} = \{x_0, x_1, x_2, \dots, x_L\}$ of length L arriving at time $t = 0$ until time $t = L$. Hence latest data is x_L and it depends on its previous data

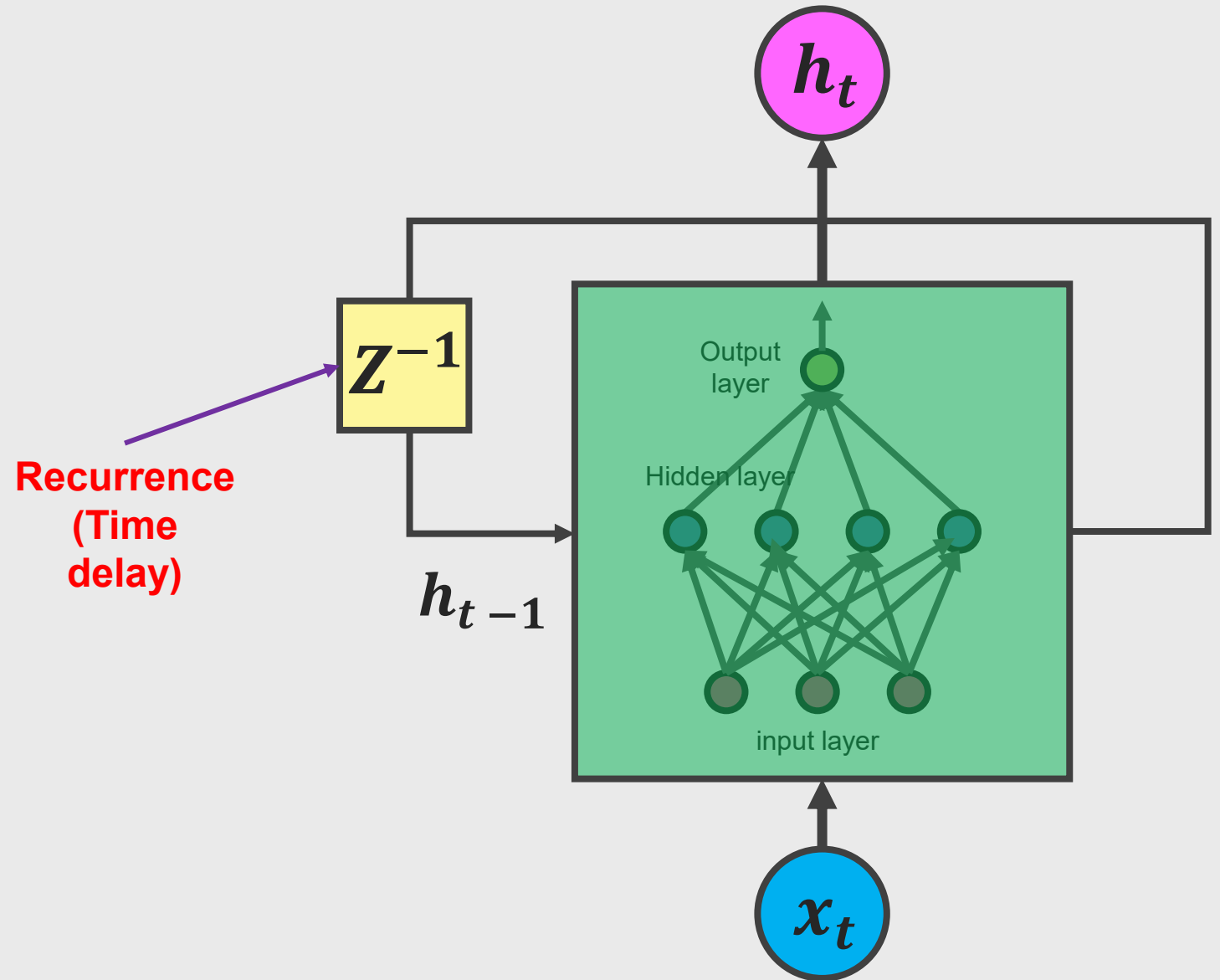
Dependency in Sequential Data



Time dependent
(sequential) data
input to a neural
network



Recurrent
Neural Network
(RNN)
Architecture



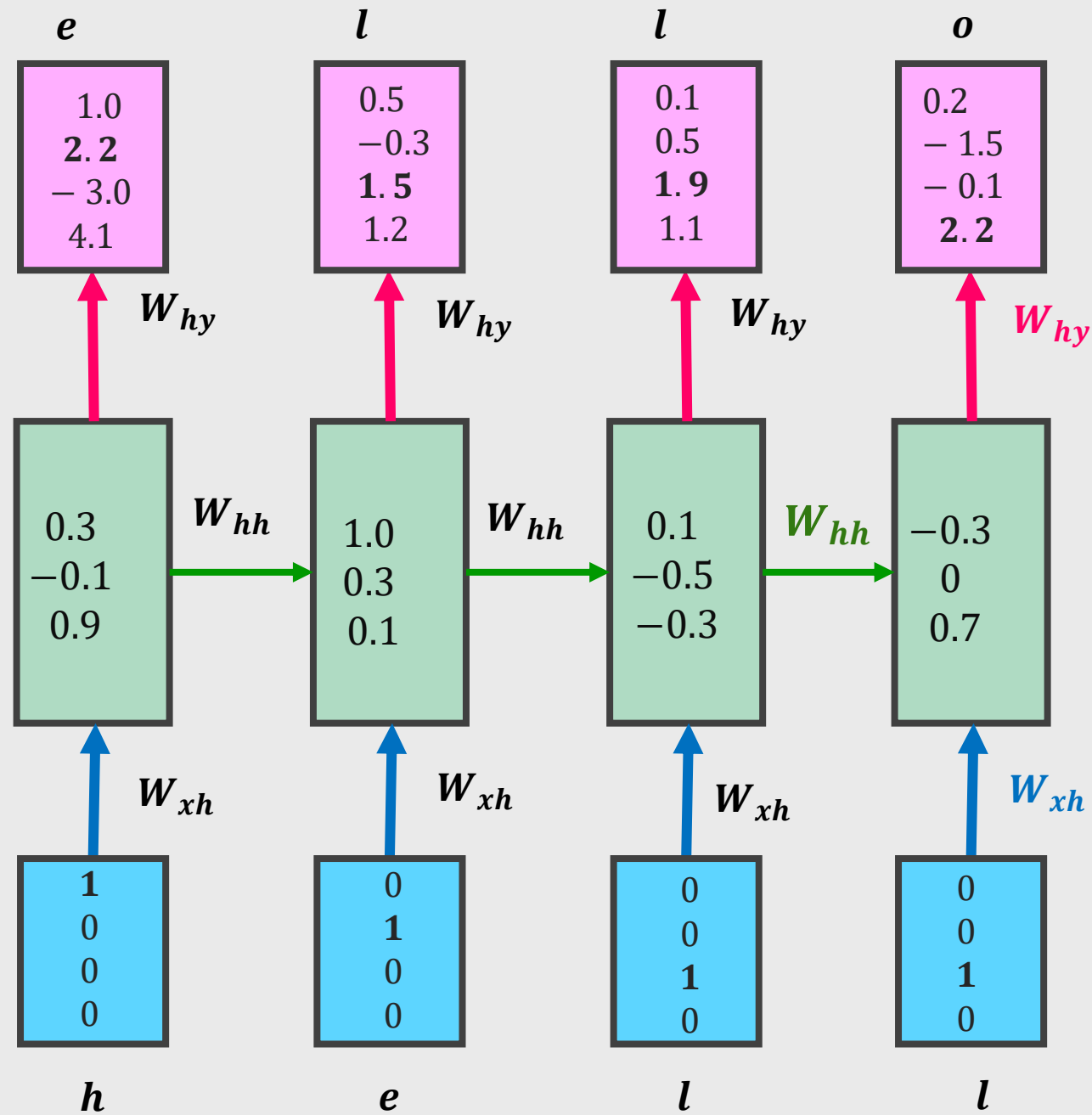
Target characters

Output layers

Hidden layer

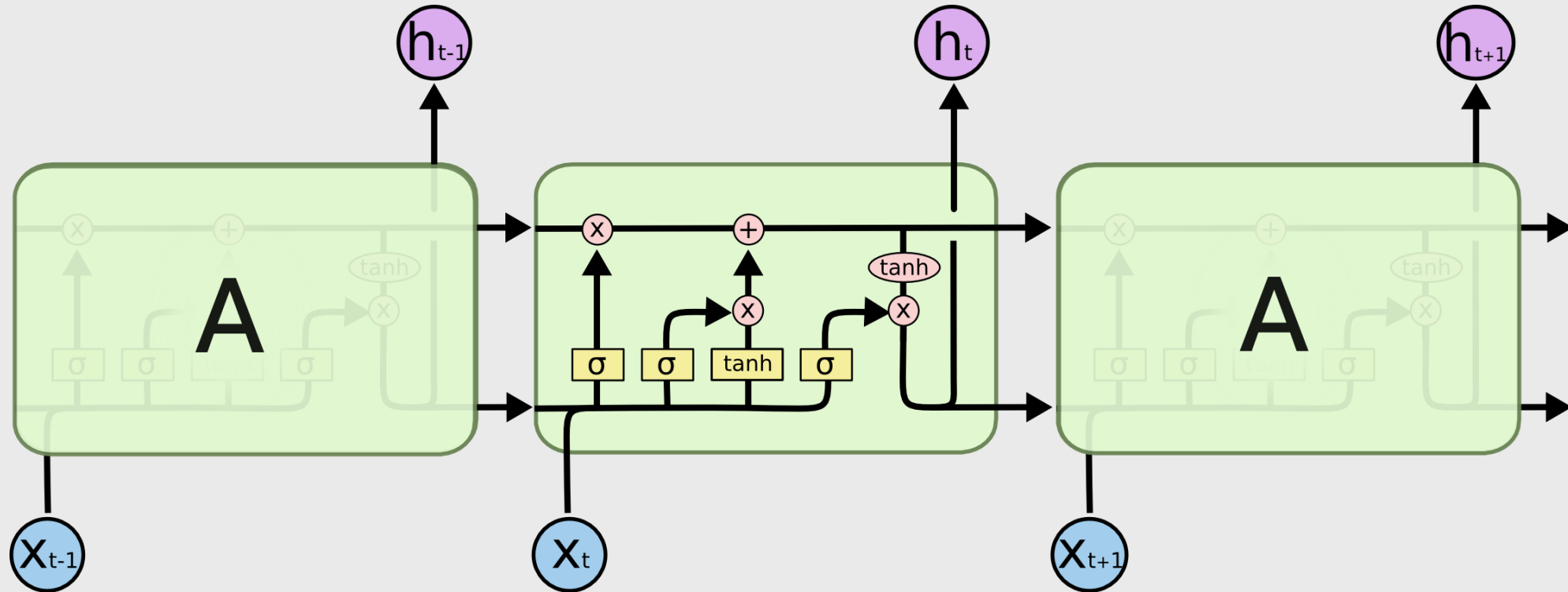
Input layer

Input characters



Long-Short Term Memory Networks

Not in the scope of this module

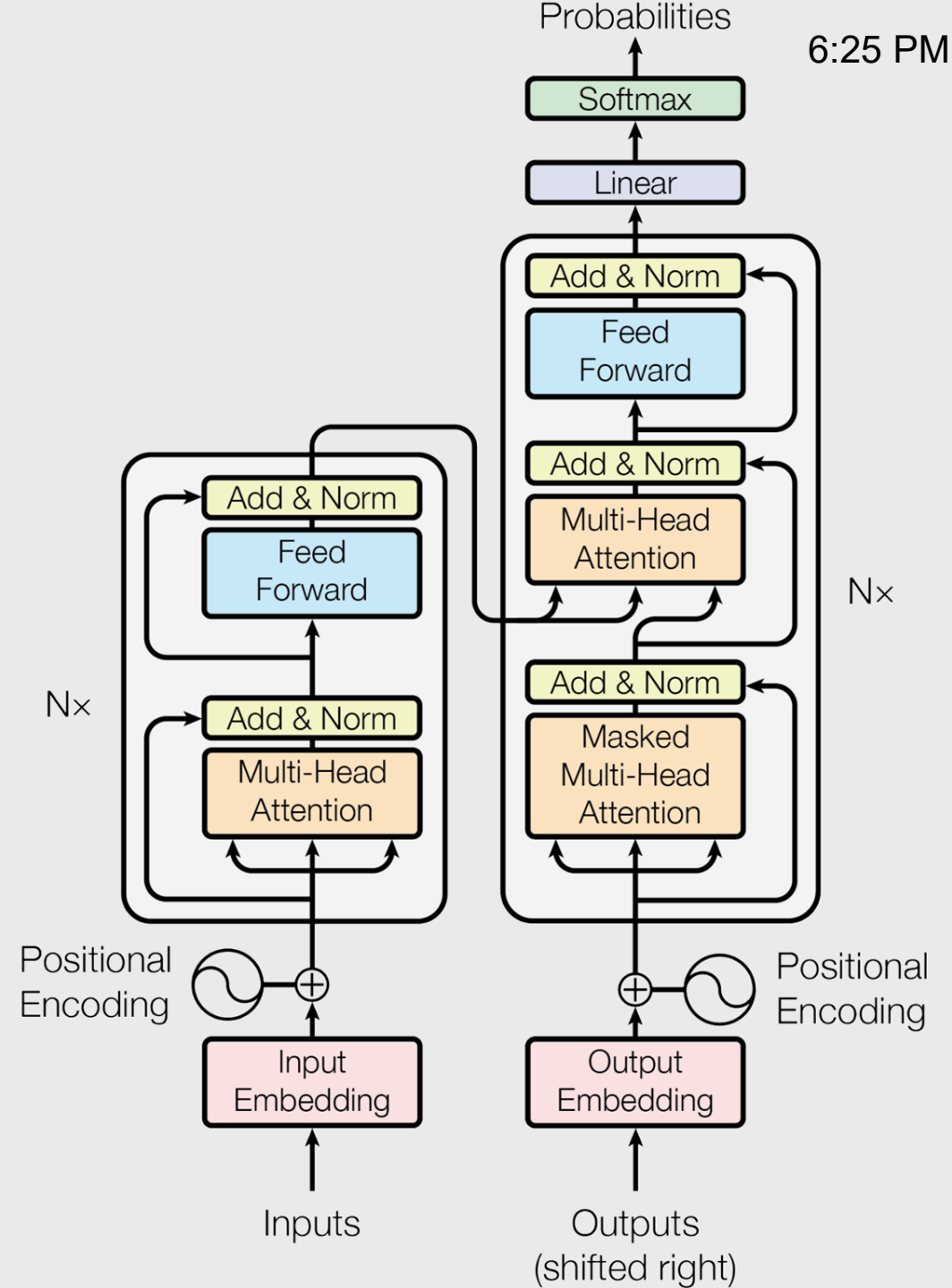


Disadvantages of Recurrent NN

- **Scalability.** Processes data sequentially, one element at a time, inherently limits parallel processing and result in slower training, especially on long sequences (e.g., processing longer sentence).
- **Dependencies.** Since RNN relies of sequential training/update of hidden states long range dependencies gets lost/distorted.
- **Context Embedding.** While RNN take care of past and future context/dependencies of words in sentence, the recurring nature of embedding word context still gets distorted as the range/length of dependencies increase.

Transformer

solves most of the
drawbacks of
Recurrent Neural
Networks



Transformer Solves Recurrent NN Drawbacks

- **Scalability.**

RNN processed data sequentially →

Transformer processed data parallelly

- **Dependencies.**

long range dependencies distortion →

Transformer positional encoding

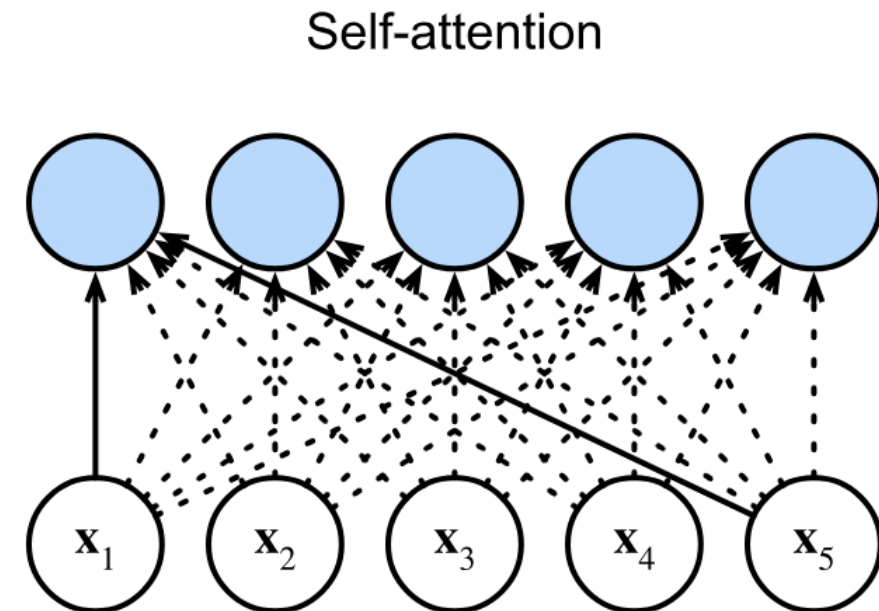
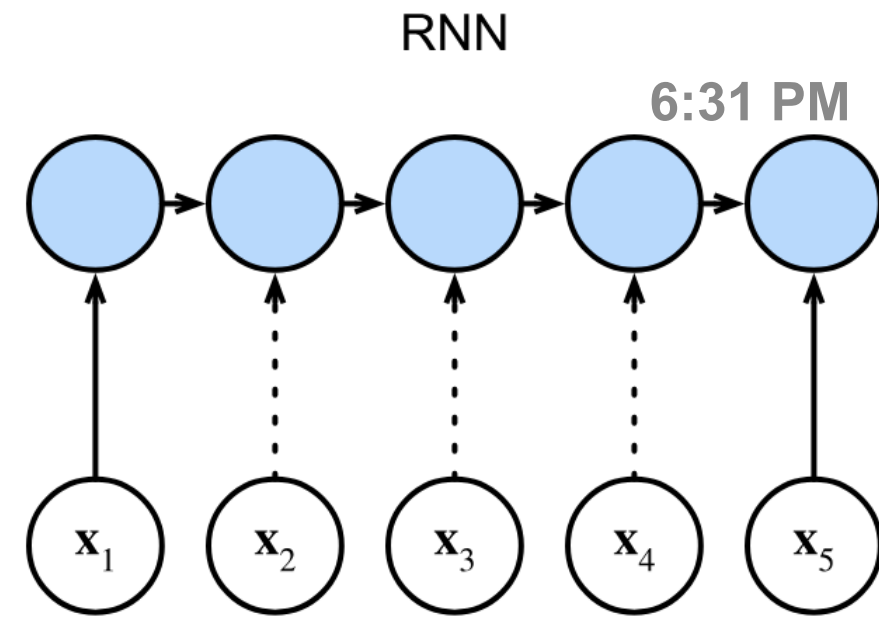
- **Context Embedding.**

RNN does context embading using recurrence →

Transformers uses self-attention

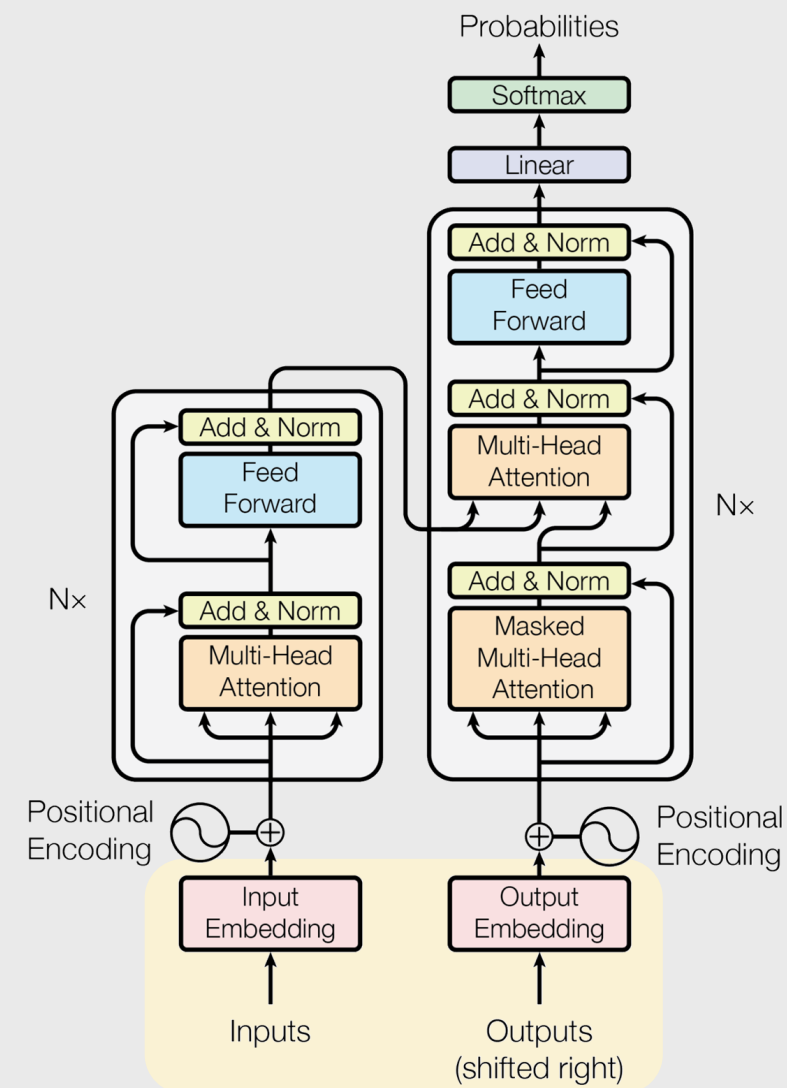
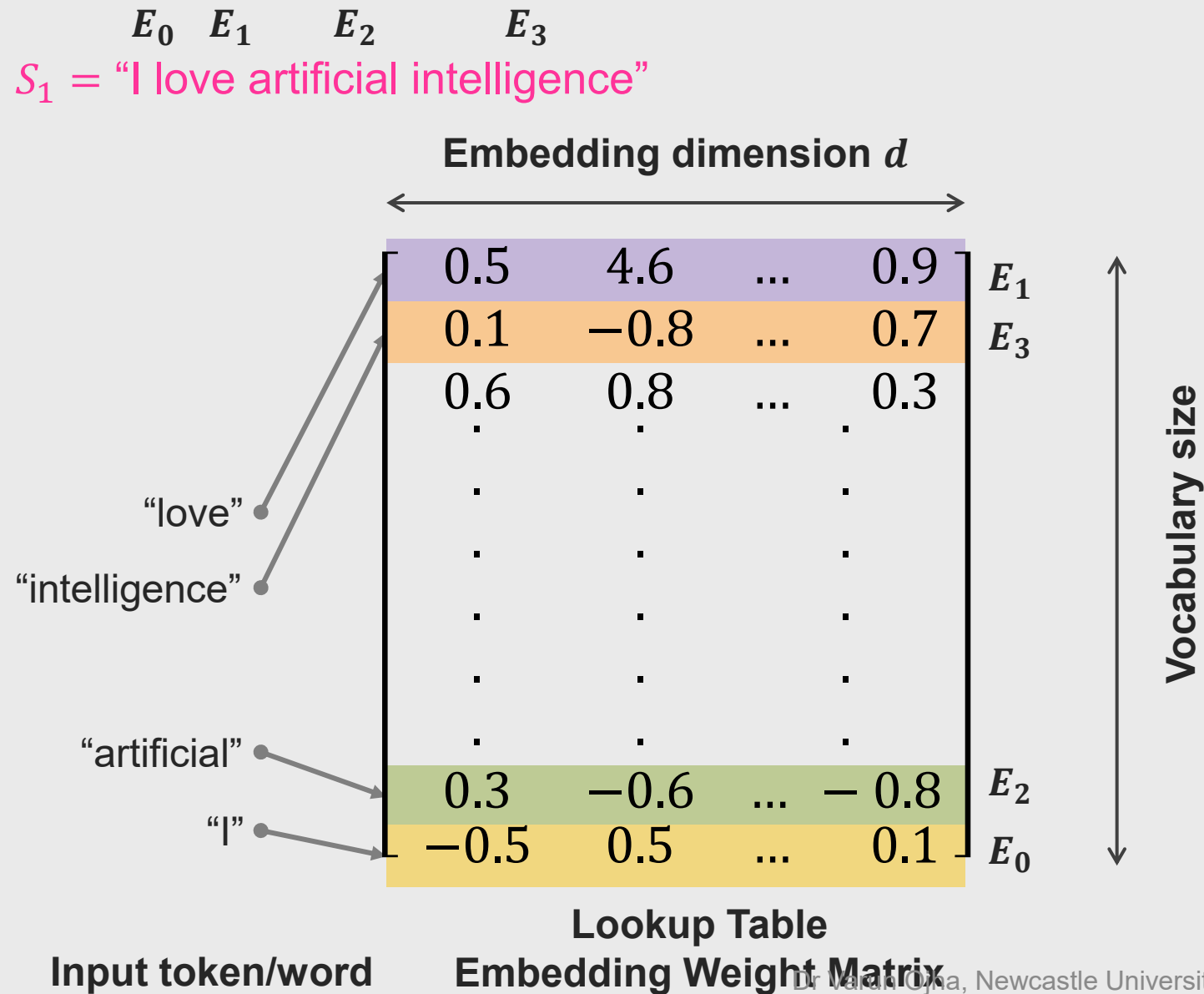
Image source: https://d2l.ai/_images/cnn-rnn-self-attention.svg

Dr Varun Ojha, Newcastle University



Word Embedding in Transformer

6:31 PM



Word Embedding in Transformer

Positional Encoding

Embedding of Word (E) Position of Word (P) Positional Embedding (PE)

$$S_1 =$$

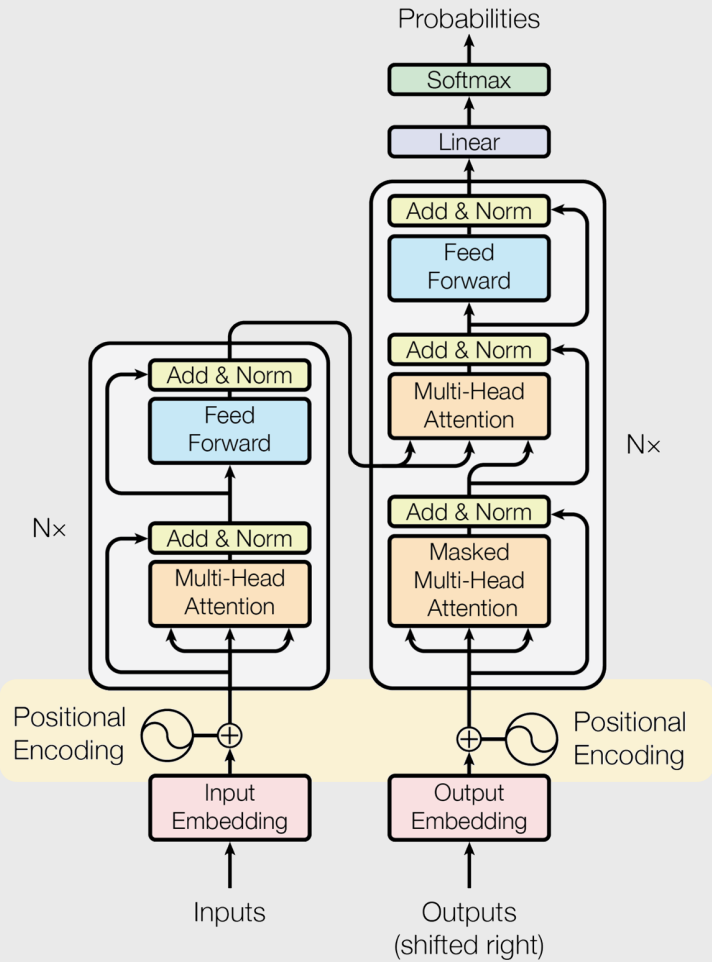
I

love

artificial

intelligence

E_0	0 th (even)	P_0	$\rightarrow$	$PE_0 = E_0 + PE_0$
E_1	1 st (odd)	P_1	$\rightarrow$	$PE_1 = E_1 + PE_1$
E_2	2 nd (even)	P_2	$\rightarrow$	$PE_2 = E_2 + PE_2$
E_3	3 rd (odd)	P_3	$\rightarrow$	$PE_3 = E_3 + PE_3$



Positional Encoding (PE)

Let's have a sentence "I love artificial intelligence" of $n = 4$ sequence length

And each word x_t (e.g., "artificial") is represented by an embedding vector of size for example $d = 10$ (this could be very large number)

That is mathematically t-th word is represented as

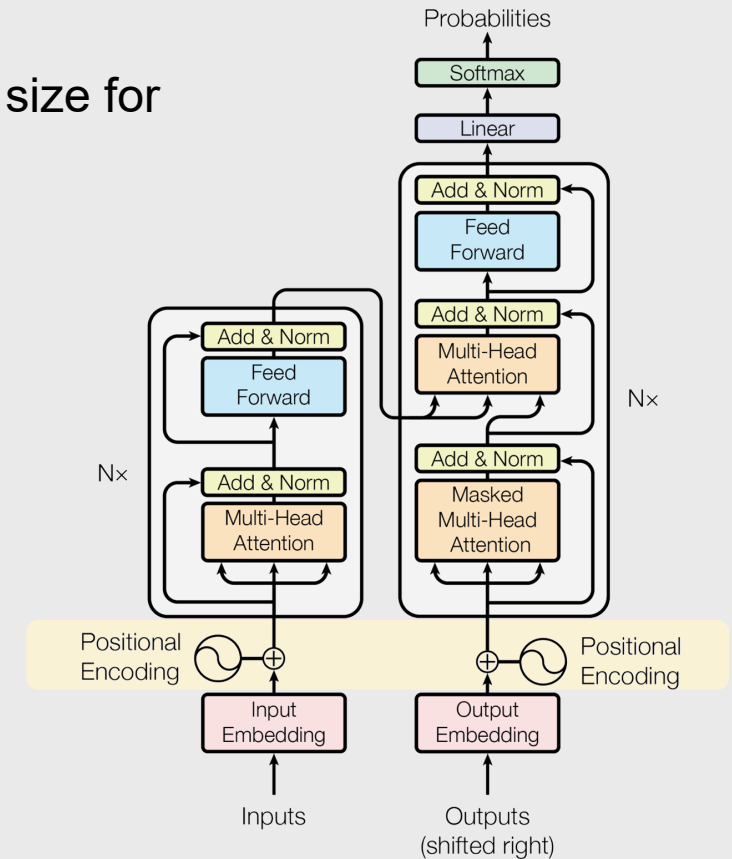
$$x_t \in \mathbb{R}^d$$

Then the positional encoding will be presented as:

$$p(pos, 2i) = \sin \left(\frac{pos}{10000^{2i/d}} \right)$$

$$p(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{2i/d}} \right)$$

For $pos = 0, 1, \dots, n$ and $i = 0, 1, \dots, \frac{d}{2}$



Positional Encoding of word: $x_t \in \mathbb{R}^d$

It assign a value relevant to the position of the word in the sentence

$$p_{pos} = \begin{bmatrix} \sin(\omega_1) \\ \cos(\omega_1) \\ \sin(\omega_2) \\ \cos(\omega_2) \\ \vdots \\ \vdots \\ \sin(\omega_{d/2}) \\ \cos(\omega_{d/2}) \end{bmatrix}$$

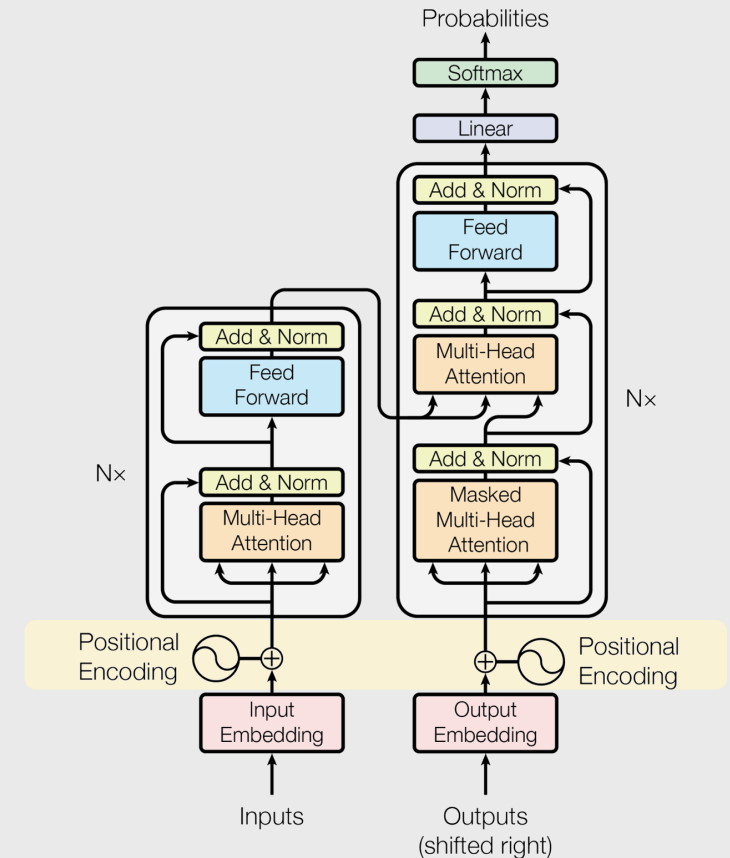
Alternating function and values

where

$$\omega_t = \frac{pos}{10000^{2i/d}}$$

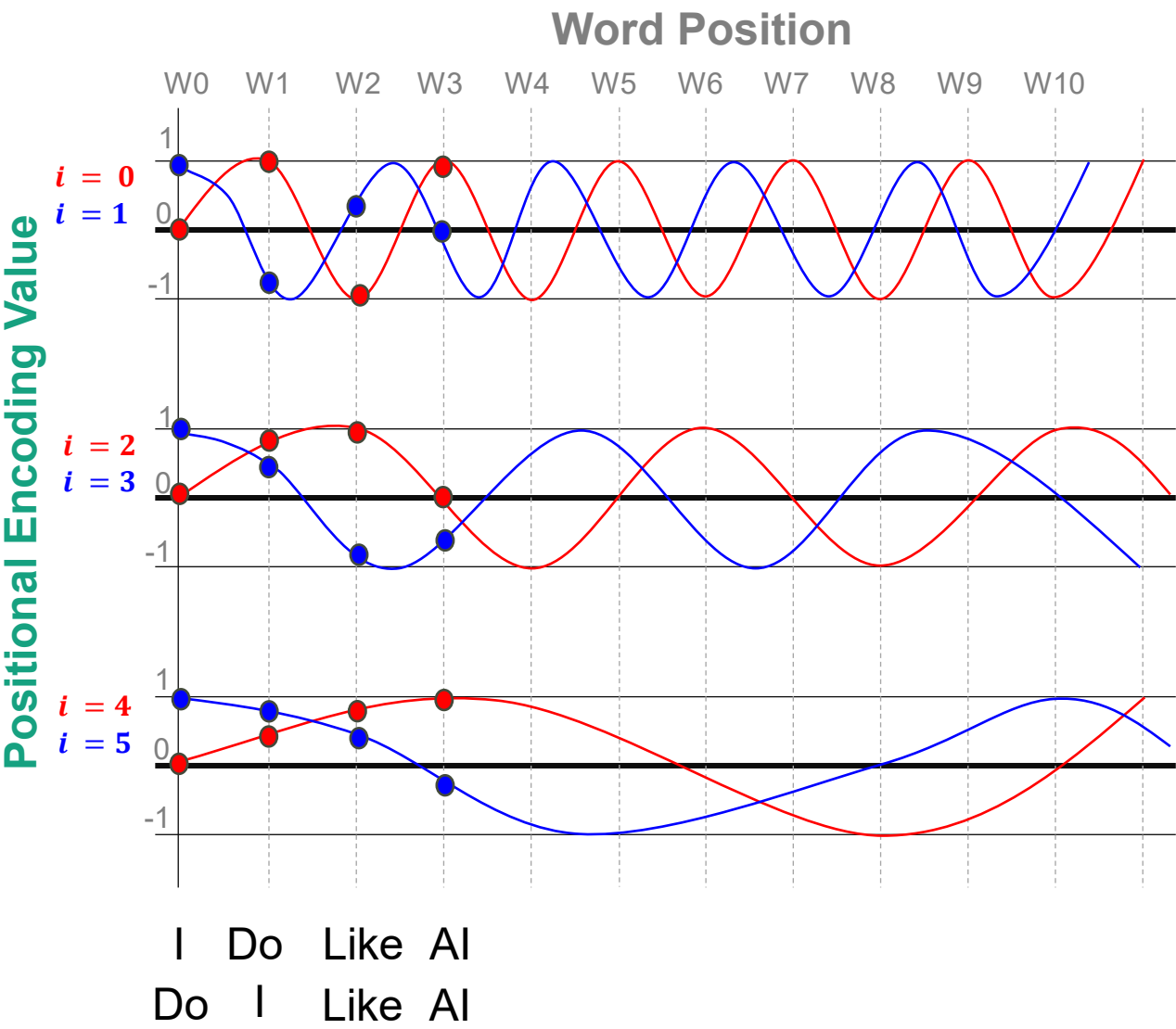
$$PE = \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_n \end{bmatrix} n \times d$$

Positional encoding matrix,
where n is sequence
length and d is embedding
dimension



Positional Encoding

It assign a value relevant to the position of the word in the sentence (say if we have two sentences of 4 words)



	I	Do	Like	AI				
	E_0	P_0	E_1	P_1	E_2	P_2	E_3	P_3
d	i							
0	0.1	0	1.5	1	3.6	-1	5.4	1
1	0.5	1	2.6	-0.9	5.7	0.5	-3.3	0
2	0.2	0	4.6	0.8	-5.5	1	-2.2	0
3	1.6	1	-1.0	0.5	-7.1	-0.8	-1.0	0.5
4	-3.5	0	-5.5	0.5	2.1	0.8	4.6	1
5	5.6	1	4.1	0.8	2.9	0.5	0.1	-0.2

	Do	I	Like	AI				
	E_1	P_0	E_0	P_1	E_2	P_2	E_3	P_3
d	i							
0	1.5	0	0.1	1	3.6	-1	5.4	1
1	2.6	1	0.5	-0.9	5.7	0.5	-3.3	0
2	4.6	0	0.2	0.8	-5.5	1	-2.2	0
3	-1.0	1	1.6	0.5	-7.1	-0.8	-1.0	0.5
4	-5.5	0	-3.5	0.5	2.1	0.8	4.6	1
5	4.1	1	5.6	0.8	2.9	0.5	0.1	-0.2

d Dimension of word embedding

i Dimension of position embedding

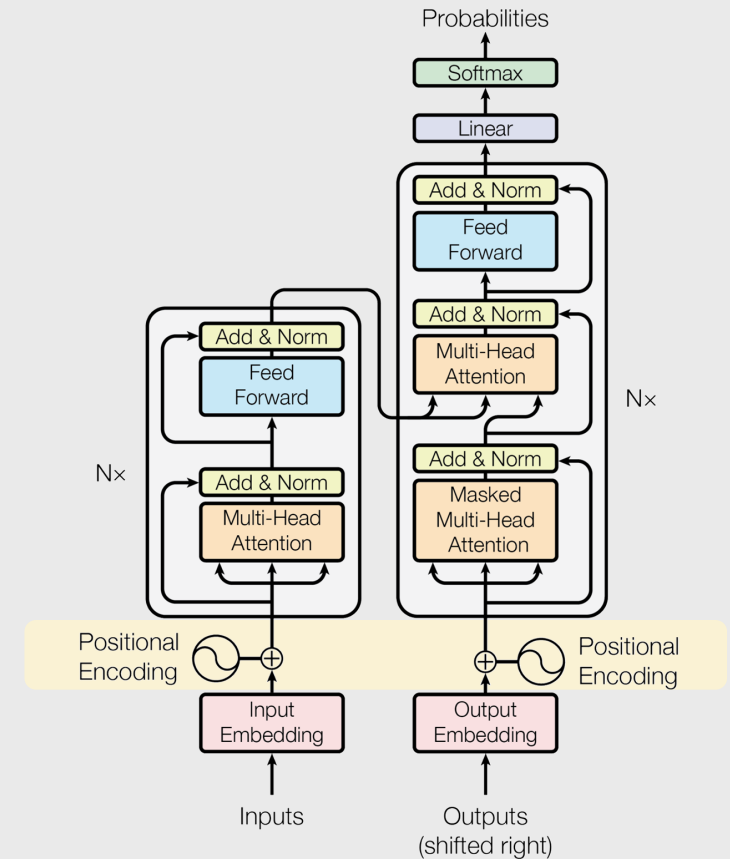
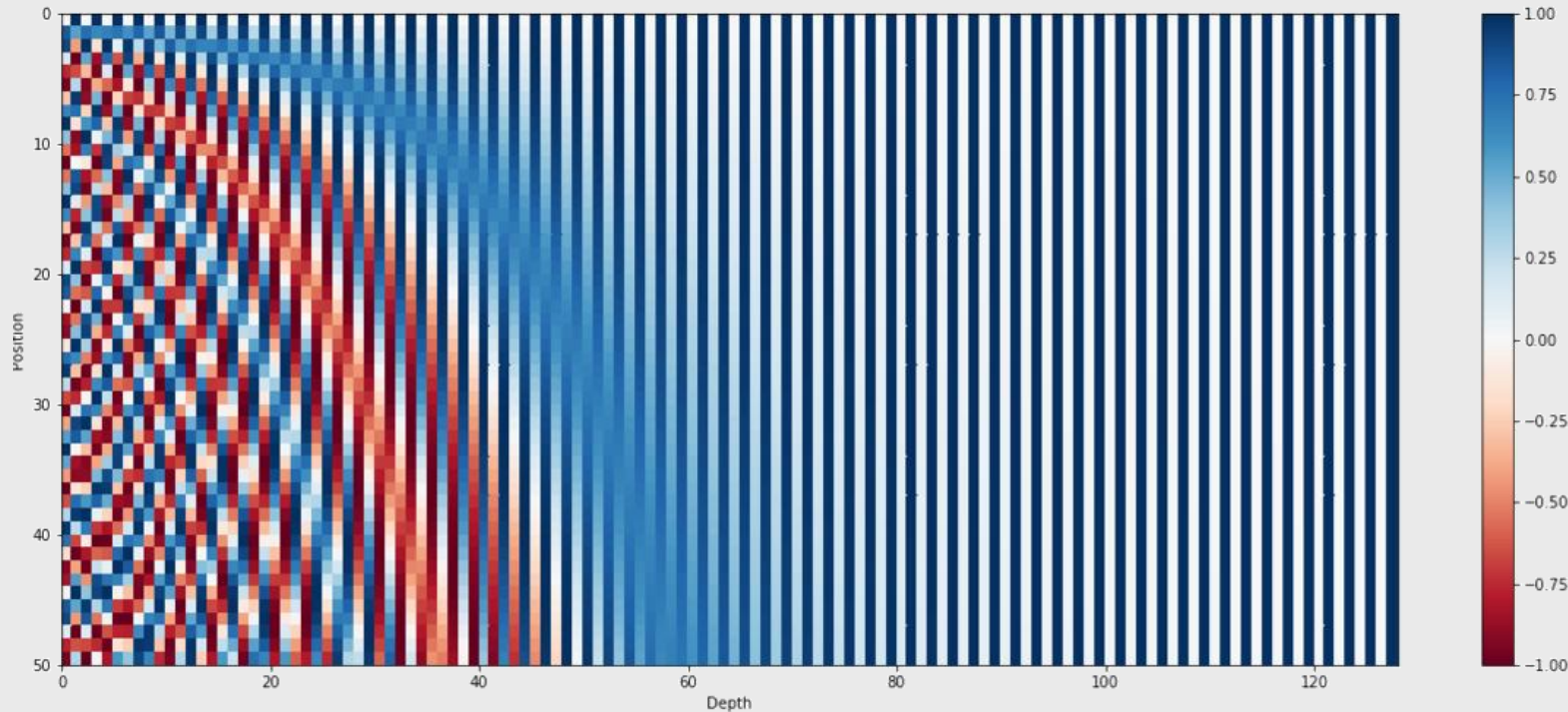
$\sin(\omega_t)$
 $\cos(\omega_t)$

Alternating function and values

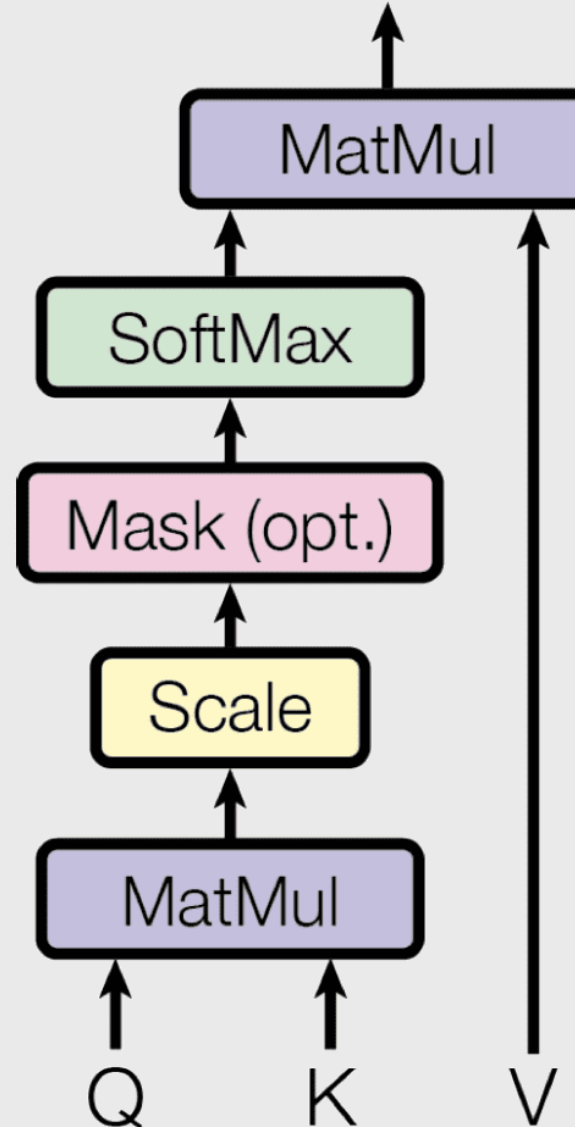
$\omega_t = \frac{pos}{10000^{2i/d}}$

Positional Encoding of word: $x_t \in \mathbb{R}^d$

It assign a value relevant to the position of the word in the sentence

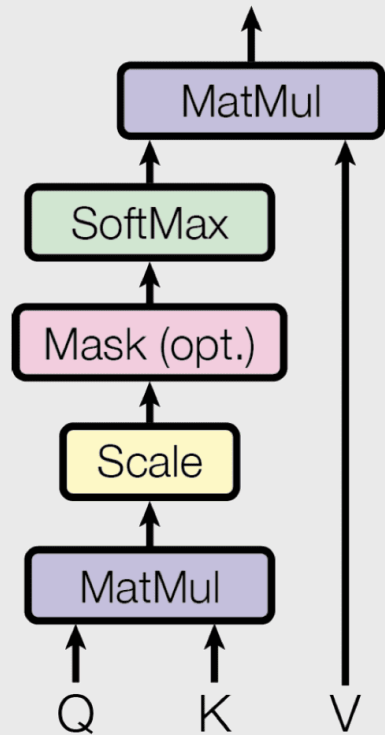


Self-Attention



Self-Attention

6:31 PM



What am I Looking for?

This is the information/answer

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

Here is what I have

Self-Attention

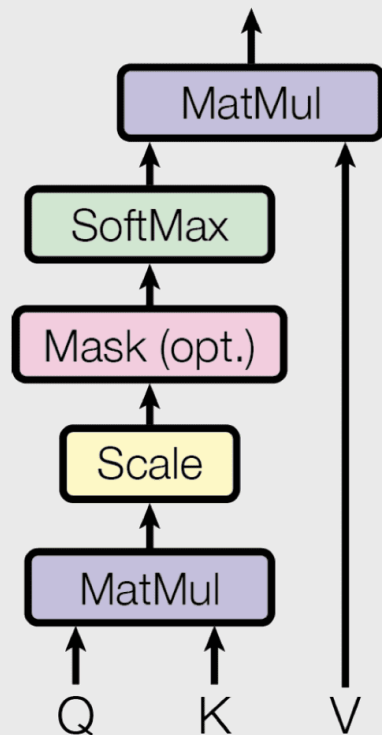
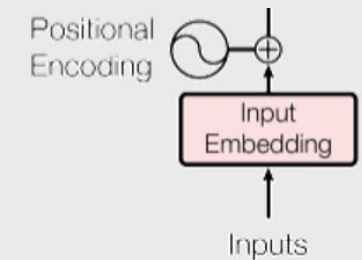
6:31 PM

Let's have a sentence "This is Computer Vision Class" of $n = 5$ sequence length

And each word x (e.g., "Computer") is represented by an embedding vector of size for example $d = 8$ (this could be very large number)

That is mathematically a word/token embedding is presented as

$$x^j \in \mathbb{R}^d$$



And we have weight matrices

$$\text{Query } \mathbf{W}_Q = d \times d_k, \quad \text{Key } \mathbf{W}_K = d \times d_k, \quad \text{Value } \mathbf{W}_V = d \times d_v$$

Then we perform a liner transformation of the input of x^j via Query, Key and Value matrices to obtain Query, Key and Value vectors as:

$$q_i^{1 \times d_k} = x_i^{1 \times d} \times \mathbf{W}_Q^{d \times d_k}, \quad k_i^{1 \times d_k} = x_i^{1 \times d} \times \mathbf{W}_K^{d \times d_k}, \quad \text{and } v_i^{1 \times d_v} = x_i^{1 \times d} \times \mathbf{W}_V^{d \times d_v}$$

For all words $i = 1, \dots, n$ in the sentence.

if you multiply an $m \times n$ matrix by an $n \times p$ matrix, the resulting matrix will have dimensions of $m \times p$: $1 \times 10 \times 10 \times 2 = 1 \times 2$.

Self-Attention

6:31 PM

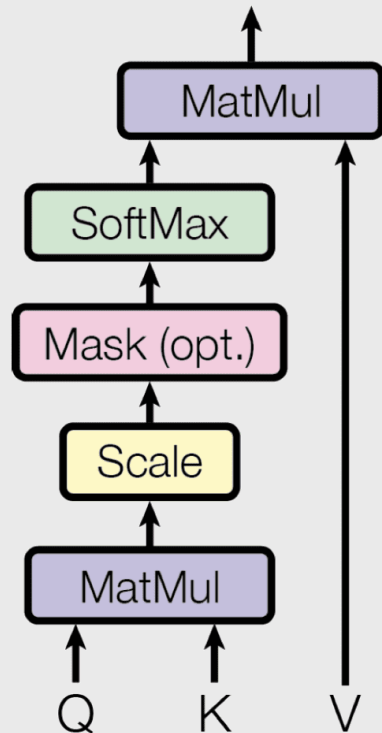
We can pack the following Query, Key and Value vectors into a matrix forms:

$$q_i^{1 \times d_k} = x_i^{1 \times d} \times W_Q^{d \times d_k}, \quad k_i^{1 \times d_k} = x_i^{1 \times d} \times W_K^{d \times d_k}, \quad \text{and} \quad v_i^{1 \times d_k} = x_i^{1 \times d} \times W_V^{d \times d_k}$$

For all words $i = 1, \dots, n$ in the sentence.

Embedding dimension of all tokens is

$$5 \times 8 \\ \mathbf{x} = [x_1, x_2, \dots, x_n]_{n \times d}$$



$$5 \times 2 \quad 5 \times 2 \quad 5 \times 2 \\ \mathbf{Q} = [q_1 \quad q_2 \quad \dots \quad q_n] \quad \mathbf{K} = [k_1 \quad k_2 \quad \dots \quad k_n] \quad \mathbf{V} = [v_1 \quad v_2 \quad \dots \quad v_n]$$

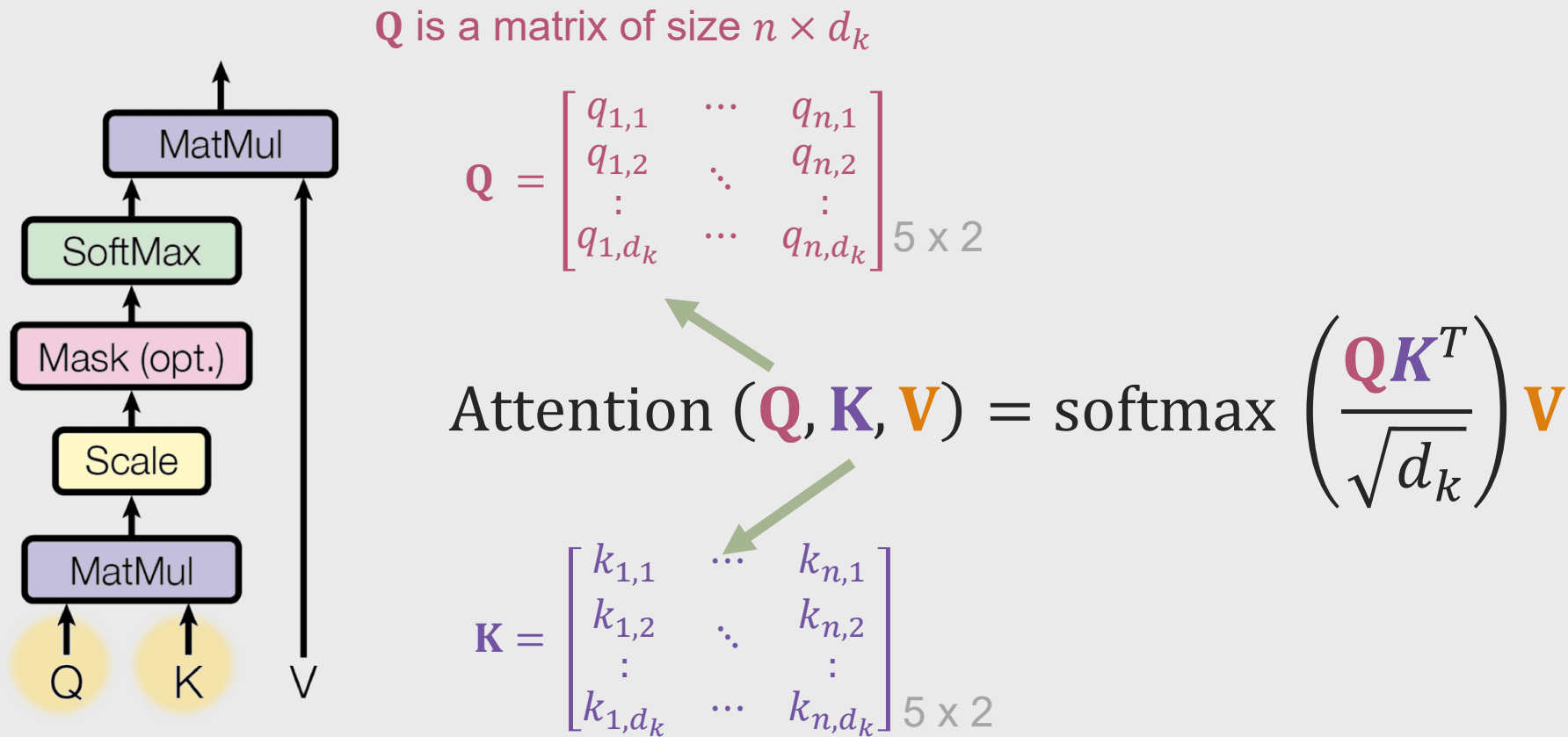
1 2 5 1 2 5 1 2 5

Self-Attention

Vaswani et al. Attention Is All You Need (NIPS 2017)

6:31 PM

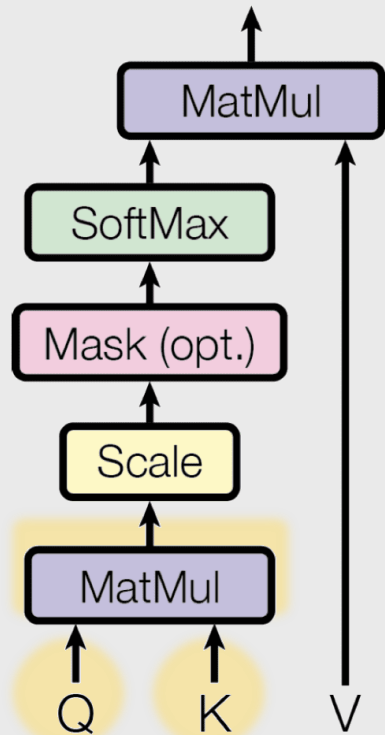
n is the number of tokens in a sentence



K is a matrix of size $n \times d_k$

Self-Attention

6:31 PM



$\mathbf{Q}$ is a matrix of size $n \times d_k$

$$\mathbf{Q} = \begin{bmatrix} q_{1,1} & \cdots & q_{1,d_k} \\ q_{2,1} & \cdots & q_{2,d_k} \\ \vdots & \ddots & \vdots \\ q_{n,1} & \cdots & q_{n,d_k} \end{bmatrix} 5 \times 2$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

$$\mathbf{K} = \begin{bmatrix} k_{1,1} & \cdots & k_{n,1} \\ k_{1,2} & \cdots & k_{n,2} \\ \vdots & \ddots & \vdots \\ k_{1,d_k} & \cdots & k_{n,d_k} \end{bmatrix} 5 \times 2$$

$\mathbf{K}$ is a matrix of size $n \times d_k$

$$\begin{bmatrix} e_{1,1} & e_{1,2} & \cdots & e_{1,n} \\ e_{2,1} & e_{2,2} & \cdots & e_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ e_{n,1} & e_{n,2} & \cdots & e_{n,n} \end{bmatrix} 5 \times 5$$

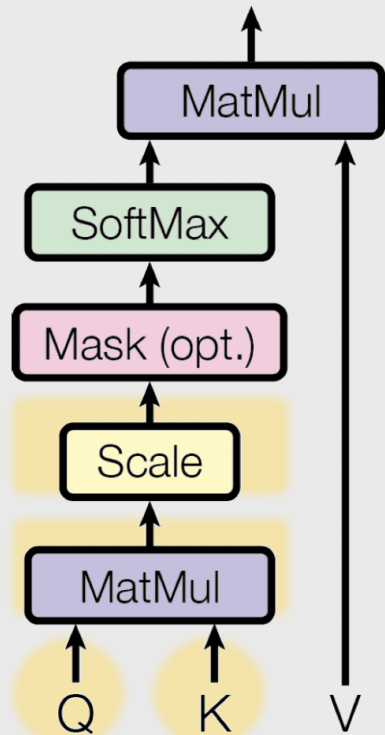
alignment scores

$$\begin{bmatrix} q_{1,1} & \cdots & q_{1,d_k} \\ q_{2,1} & \cdots & q_{2,d_k} \\ \vdots & \ddots & \vdots \\ q_{n,1} & \cdots & q_{n,d_k} \end{bmatrix} 5 \times 2 \cdot \begin{bmatrix} k_{1,1} & \cdots & k_{n,1} \\ k_{1,2} & \cdots & k_{n,2} \\ \vdots & \ddots & \vdots \\ k_{1,d_k} & \cdots & k_{n,d_k} \end{bmatrix} 2 \times 5$$

dot product

Self-Attention

6:31 PM



$\mathbf{Q}$ is a matrix of size $n \times d_k$

$$\mathbf{Q} = \begin{bmatrix} q_{1,1} & \cdots & q_{n,1} \\ q_{1,2} & \ddots & q_{n,2} \\ \vdots & & \vdots \\ q_{1,d_k} & \cdots & q_{n,d_k} \end{bmatrix} 5 \times 2$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

$$\mathbf{K} = \begin{bmatrix} k_{1,1} & \cdots & k_{n,1} \\ k_{1,2} & \ddots & k_{n,2} \\ \vdots & & \vdots \\ k_{1,d_k} & \cdots & k_{n,d_k} \end{bmatrix} 5 \times 2$$

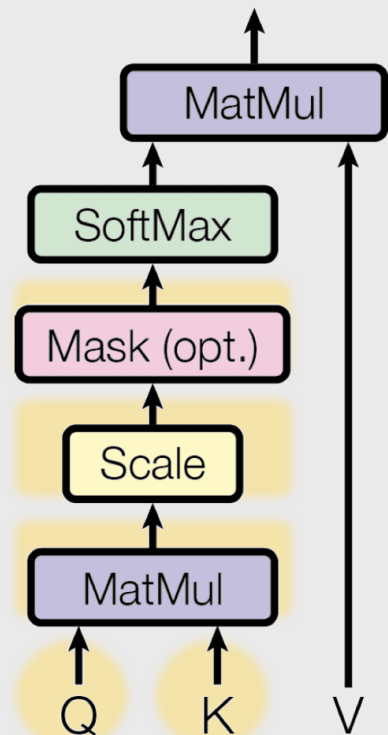
$\mathbf{K}$ is a matrix of size $n \times d_k$

$$\begin{bmatrix} \frac{e_{1,1}}{\sqrt{d_k}} & \frac{e_{1,2}}{\sqrt{d_k}} & \cdots & \frac{e_{1,n}}{\sqrt{d_k}} \\ \frac{e_{2,1}}{\sqrt{d_k}} & \frac{e_{2,2}}{\sqrt{d_k}} & \cdots & \frac{e_{2,n}}{\sqrt{d_k}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{e_{n,1}}{\sqrt{d_k}} & \frac{e_{n,2}}{\sqrt{d_k}} & \cdots & \frac{e_{n,n}}{\sqrt{d_k}} \end{bmatrix} 5 \times 5$$

Scaling of alignment scores

Self-Attention

6:31 PM



Q is a matrix of size $n \times d_k$

$$Q = \begin{bmatrix} q_{1,1} & \cdots & q_{n,1} \\ q_{1,2} & \ddots & q_{n,2} \\ \vdots & & \vdots \\ q_{1,d_k} & \cdots & q_{n,d_k} \end{bmatrix} 5 \times 2$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$K = \begin{bmatrix} k_{1,1} & \cdots & k_{n,1} \\ k_{1,2} & \ddots & k_{n,2} \\ \vdots & & \vdots \\ k_{1,d_k} & \cdots & k_{n,d_k} \end{bmatrix} 5 \times 2$$

K is a matrix of size $n \times d_k$

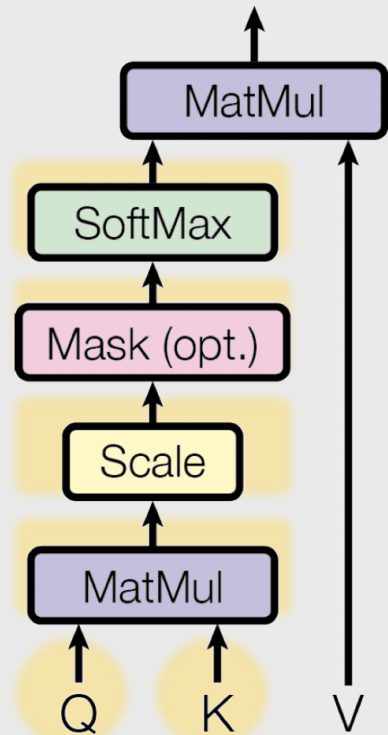
$$\begin{bmatrix} \frac{e_{1,1}}{\sqrt{d_k}} & -\infty & -\infty \\ \frac{e_{2,1}}{\sqrt{d_k}} & \frac{e_{2,2}}{\sqrt{d_k}} & \cdots & -\infty \\ \vdots & \vdots & \ddots & \vdots \\ \frac{e_{n,1}}{\sqrt{d_k}} & \frac{e_{n,2}}{\sqrt{d_k}} & \cdots & \frac{e_{n,n}}{\sqrt{d_k}} \end{bmatrix} 5 \times 5$$

This forces the model to learn to predict the next token based only on past and current tokens

Apply Mask (optional)

Self-Attention

6:31 PM



Q is a matrix of size $n \times d_k$

$$Q = \begin{bmatrix} q_{1,1} & \cdots & q_{n,1} \\ q_{1,2} & \ddots & q_{n,2} \\ \vdots & & \vdots \\ q_{1,d_k} & \cdots & q_{n,d_k} \end{bmatrix} 5 \times 2$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$K = \begin{bmatrix} k_{1,1} & \cdots & k_{n,1} \\ k_{1,2} & \ddots & k_{n,2} \\ \vdots & & \vdots \\ k_{1,d_k} & \cdots & k_{n,d_k} \end{bmatrix} 5 \times 2$$

K is a matrix of size $n \times d_k$

$$\begin{bmatrix} \text{softmax} \left(\frac{e_{1,1}}{\sqrt{d_k}} \right) & 0 & \cdots & 0 \\ \text{softmax} \left(\frac{e_{2,1}}{\sqrt{d_k}} \right) & \frac{e_{2,2}}{\sqrt{d_k}} & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ \text{softmax} \left(\frac{e_{n,1}}{\sqrt{d_k}} \right) & \frac{e_{n,2}}{\sqrt{d_k}} & \cdots & \frac{e_{n,n}}{\sqrt{d_k}} \end{bmatrix} 5 \times 5$$

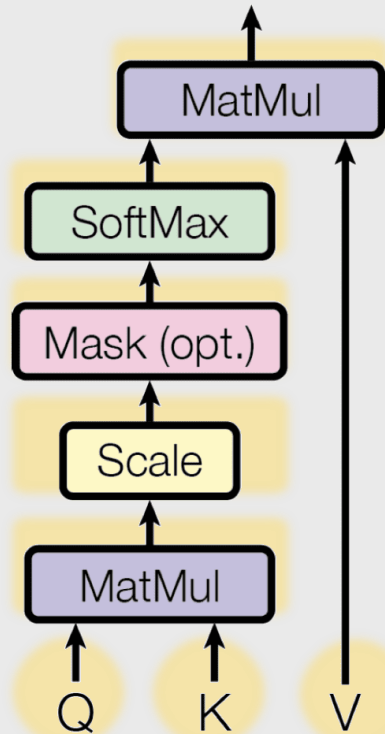
SoftMax of Scaled dot product

$\frac{1}{\sqrt{d_k}}$ is the scaling factor

Self-Attention

Vaswani et al. Attention Is All You Need (NIPS 2017)

6:31 PM



$\mathbf{Q}$ is a matrix of size $n \times d_k$

$$\mathbf{Q} = \begin{bmatrix} q_{1,1} & \cdots & q_{n,1} \\ q_{1,2} & \ddots & q_{n,2} \\ \vdots & & \vdots \\ q_{1,d_k} & \cdots & q_{n,d_k} \end{bmatrix} 5 \times 2$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

$$\mathbf{K} = \begin{bmatrix} k_{1,1} & \cdots & k_{n,1} \\ k_{1,2} & \ddots & k_{n,2} \\ \vdots & & \vdots \\ k_{1,d_k} & \cdots & k_{n,d_k} \end{bmatrix} 5 \times 2$$

$$\mathbf{V} = \begin{bmatrix} v_{1,1} & \cdots & v_{1,d_v} \\ v_{2,1} & \ddots & v_{2,d_v} \\ \vdots & & \vdots \\ v_{n,1} & \cdots & v_{n,d_v} \end{bmatrix} 5 \times 2$$

$\mathbf{K}$ is a matrix of size $n \times d_k$

$\mathbf{V}$ is a matrix of size $n \times d_v$

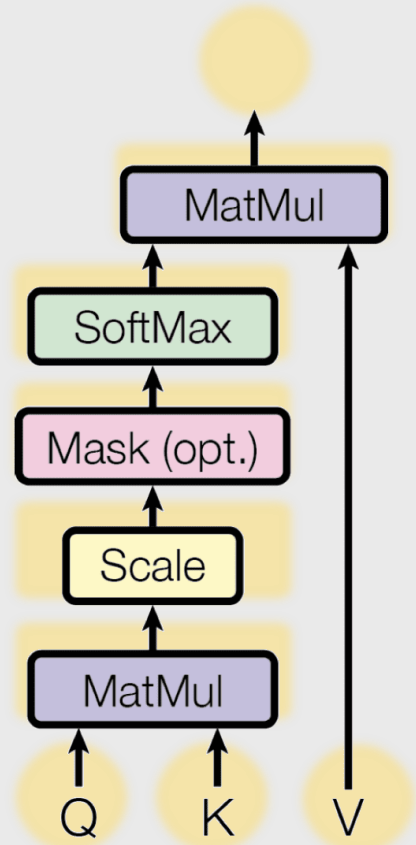
$$\begin{bmatrix} \text{softmax} \left(\frac{e_{1,1}}{\sqrt{d_k}} \right) & 0 & \cdots & 0 \\ \text{softmax} \left(\frac{e_{2,1}}{\sqrt{d_k}} \right) & \frac{e_{2,2}}{\sqrt{d_k}} & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ \text{softmax} \left(\frac{e_{d_k,1}}{\sqrt{d_k}} \right) & \frac{e_{d_k,2}}{\sqrt{d_k}} & \cdots & \frac{e_{d_k,d_k}}{\sqrt{d_k}} \end{bmatrix} 5 \times 5 \cdot \begin{bmatrix} v_{1,1} & \cdots & v_{1,d_v} \\ v_{2,1} & \ddots & v_{2,d_v} \\ \vdots & & \vdots \\ v_{n,1} & \cdots & v_{n,d_v} \end{bmatrix} 5 \times 2$$

resulting self-attention weights

Self-Attention

Vaswani et al. Attention Is All You Need (NIPS 2017)

6:31 PM



Q is a matrix of size $n \times d_k$

$$Q = \begin{bmatrix} q_{1,1} & \cdots & q_{n,1} \\ q_{1,2} & \ddots & q_{n,2} \\ \vdots & & \vdots \\ q_{1,d_k} & \cdots & q_{n,d_k} \end{bmatrix}$$

$$\text{Head}_1 = \begin{bmatrix} a_{1,1} & \cdots & a_{1,d_v} \\ a_{2,1} & \ddots & a_{2,d_v} \\ \vdots & & \vdots \\ a_{n,1} & \cdots & a_{n,d_v} \end{bmatrix} 5 \times 2$$

resulting self-attention weights

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$$K = \begin{bmatrix} k_{1,1} & \cdots & k_{n,1} \\ k_{1,2} & \ddots & k_{n,2} \\ \vdots & & \vdots \\ k_{1,d_k} & \cdots & k_{n,d_k} \end{bmatrix}$$

$$V = \begin{bmatrix} v_{1,1} & \cdots & v_{1,d_v} \\ v_{2,1} & \ddots & v_{2,d_v} \\ \vdots & & \vdots \\ v_{d,1} & \cdots & v_{d,d_v} \end{bmatrix}$$

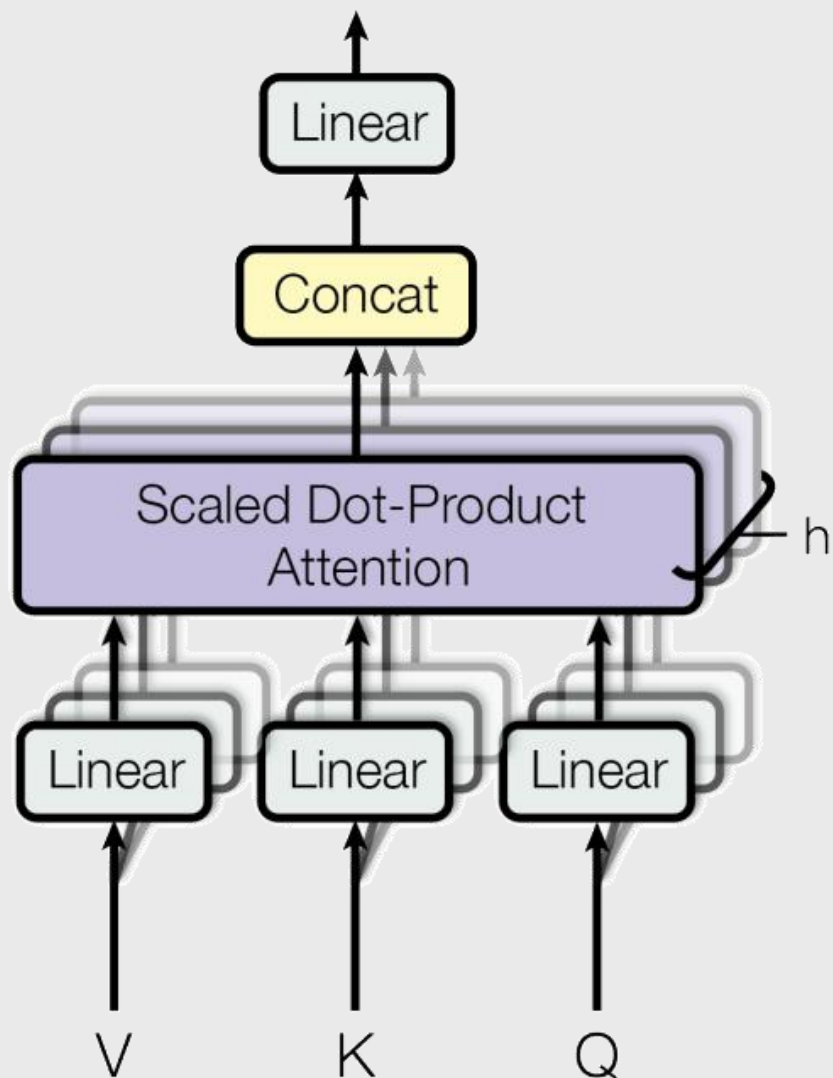
K is a matrix of size $n \times d_k$

V is a matrix of size $d \times d_v$

Multi Head Self-Attention

6:31 PM

Vaswani et al. Attention Is All You Need (NIPS 2017)



5×8

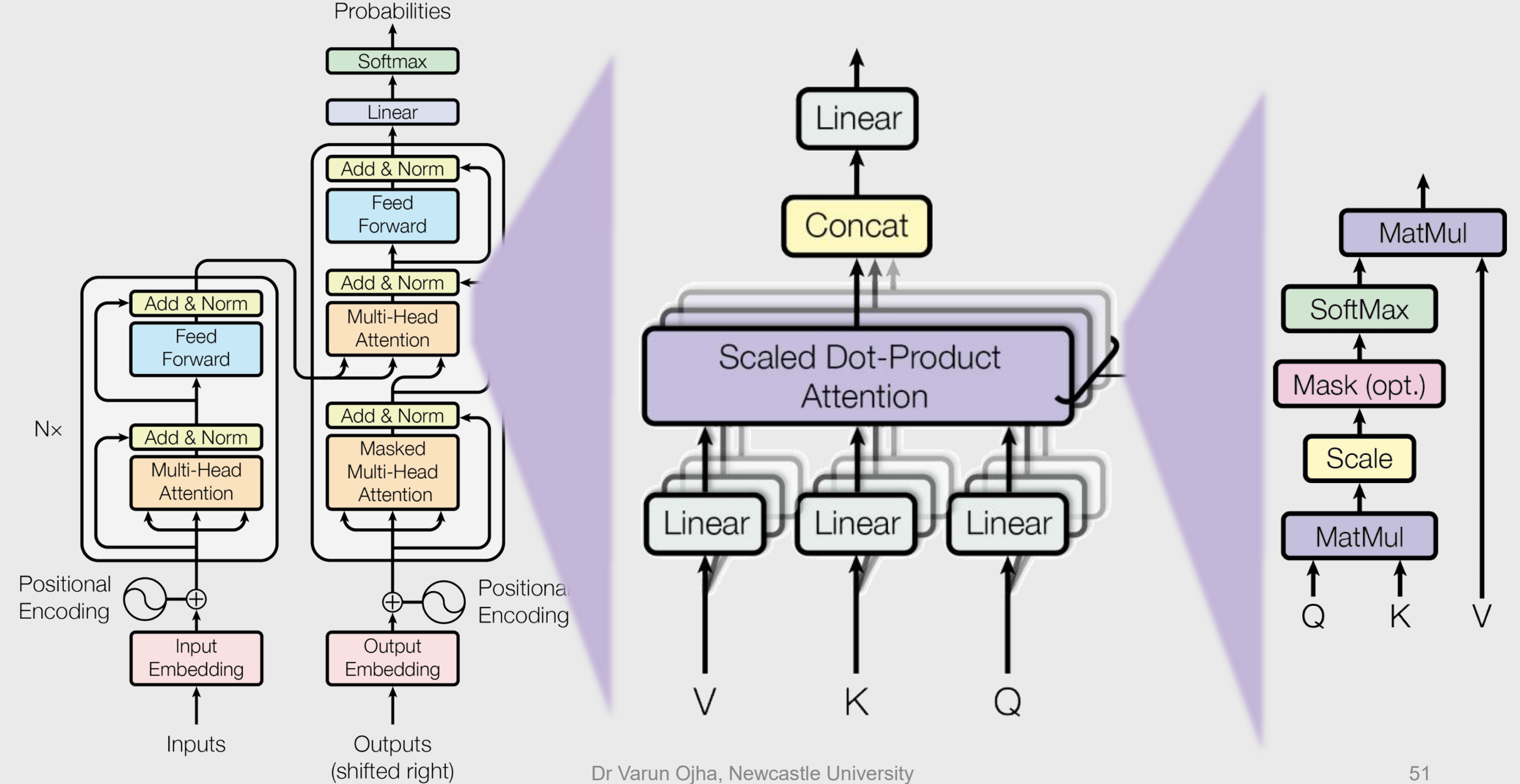
$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$

$= \text{Concat}(\text{Head}_1, \text{Head}_2, \dots, \text{Head}_h) \mathbf{W}_{\text{Out}}$

5×8 8×8

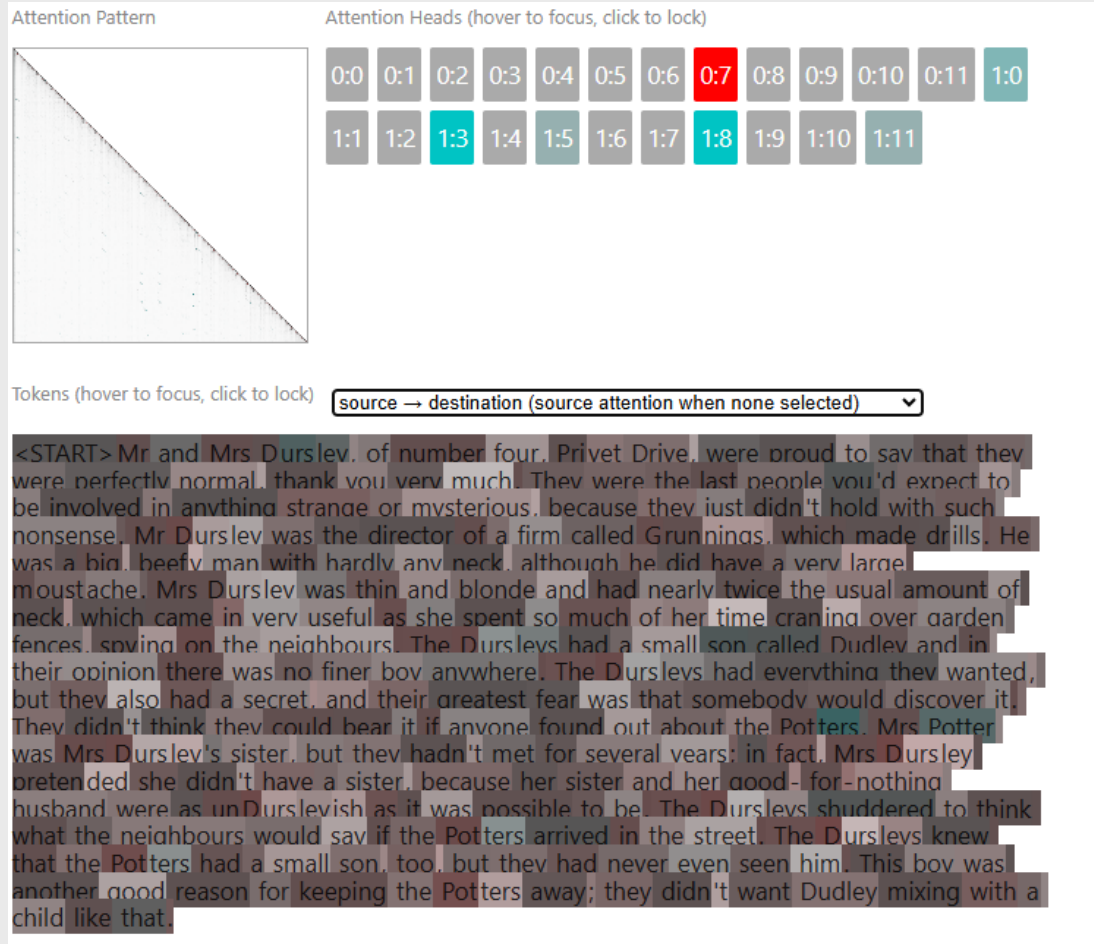
$$\begin{bmatrix} a_{1,1} & \cdots & a_{1,d_v} & a_{1,1} & \cdots & a_{1,d_v} & a_{1,1} & \cdots & a_{1,d_v} \\ a_{2,1} & \ddots & a_{2,d_v} & a_{2,1} & \ddots & a_{2,d_v} & a_{2,1} & \ddots & a_{2,d_v} \\ \vdots & \cdots & \vdots & \vdots & \cdots & \vdots & \vdots & \cdots & \vdots \\ a_{n,1} & \cdots & a_{n,d_v} & a_{n,1} & \cdots & a_{n,d_v} & a_{n,1} & \cdots & a_{n,d_v} \end{bmatrix} \mathbf{W}_{\text{Out}}$$

5×2 5×2 5×2





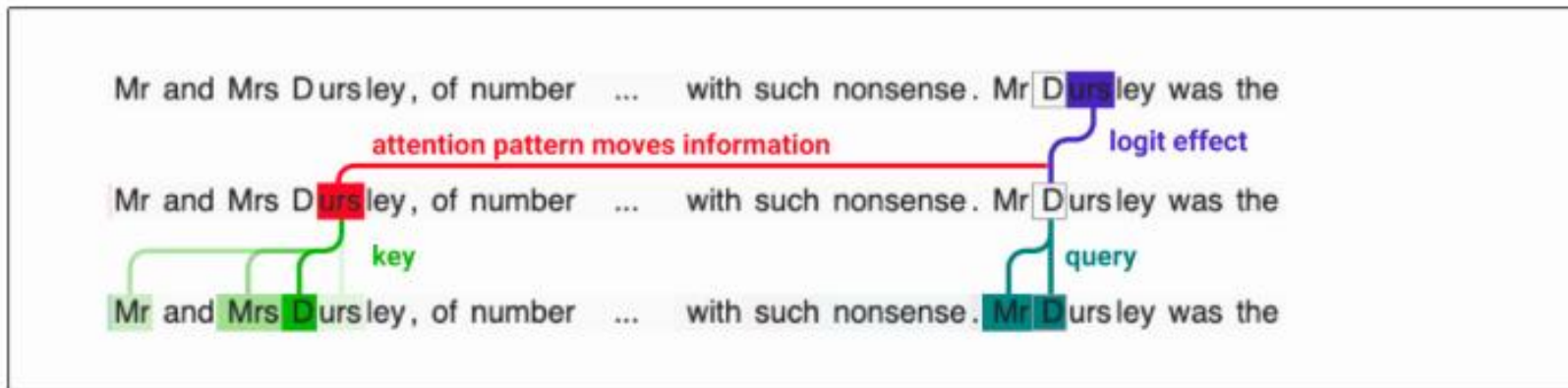
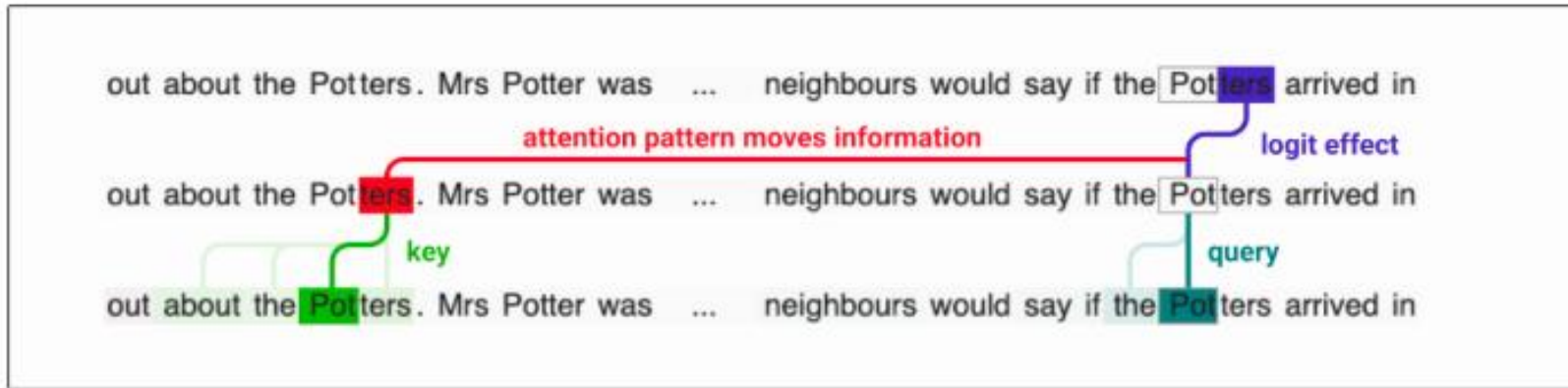
Attention Map



A Mathematical Framework for Transformer Circuits <https://transformer-circuits.pub/2021/framework/index.html>

How Query and Key might work

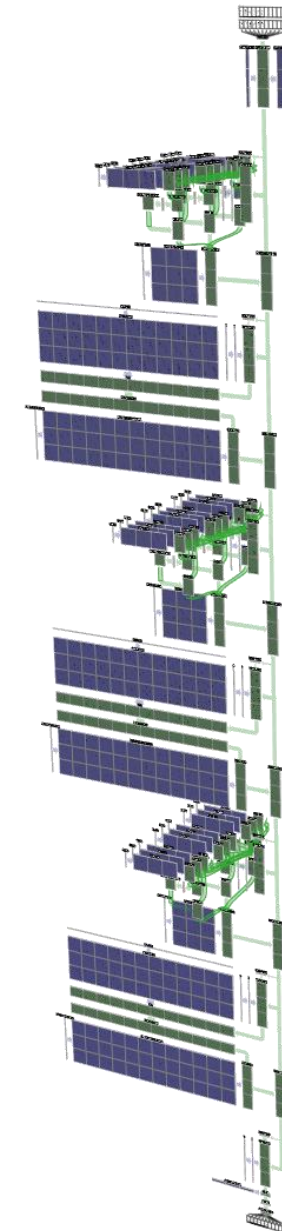
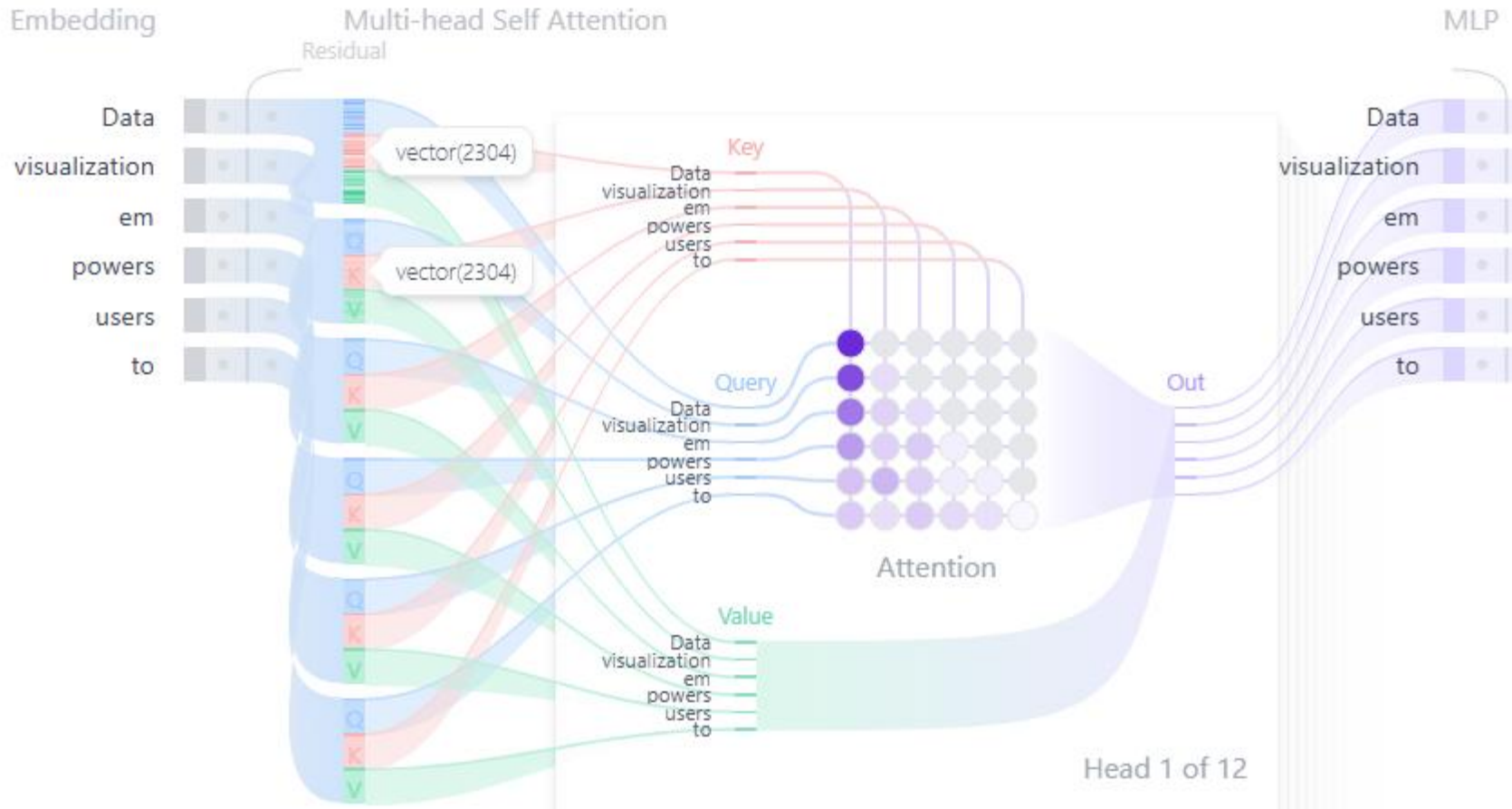
The query searches for "similar" key vectors, but because keys are shifted, it finds the next token.



Transformer Visualisations & Explainers (Online Resources)

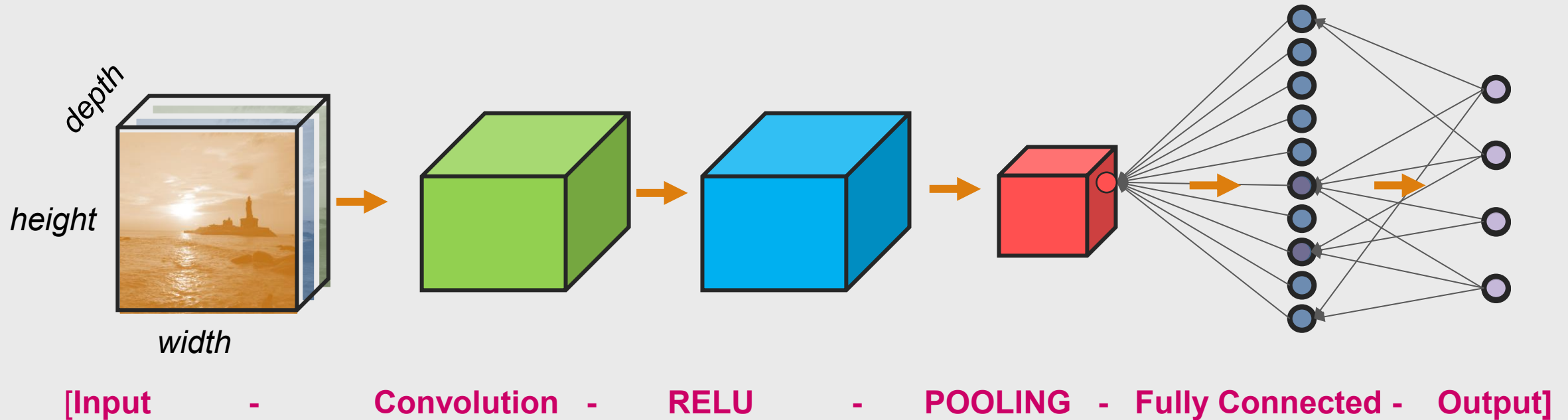
nano-gpt

n_params = 85,584 6:31 PM



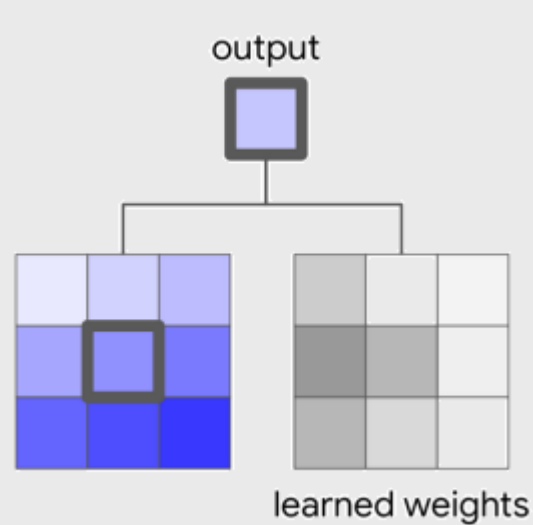
- <https://bbycroft.net/llm>
- <https://poloclub.github.io/transformer-explainer/>
- <https://jalammar.github.io/illustrated-transformer/>

Convolutional Neural Nets



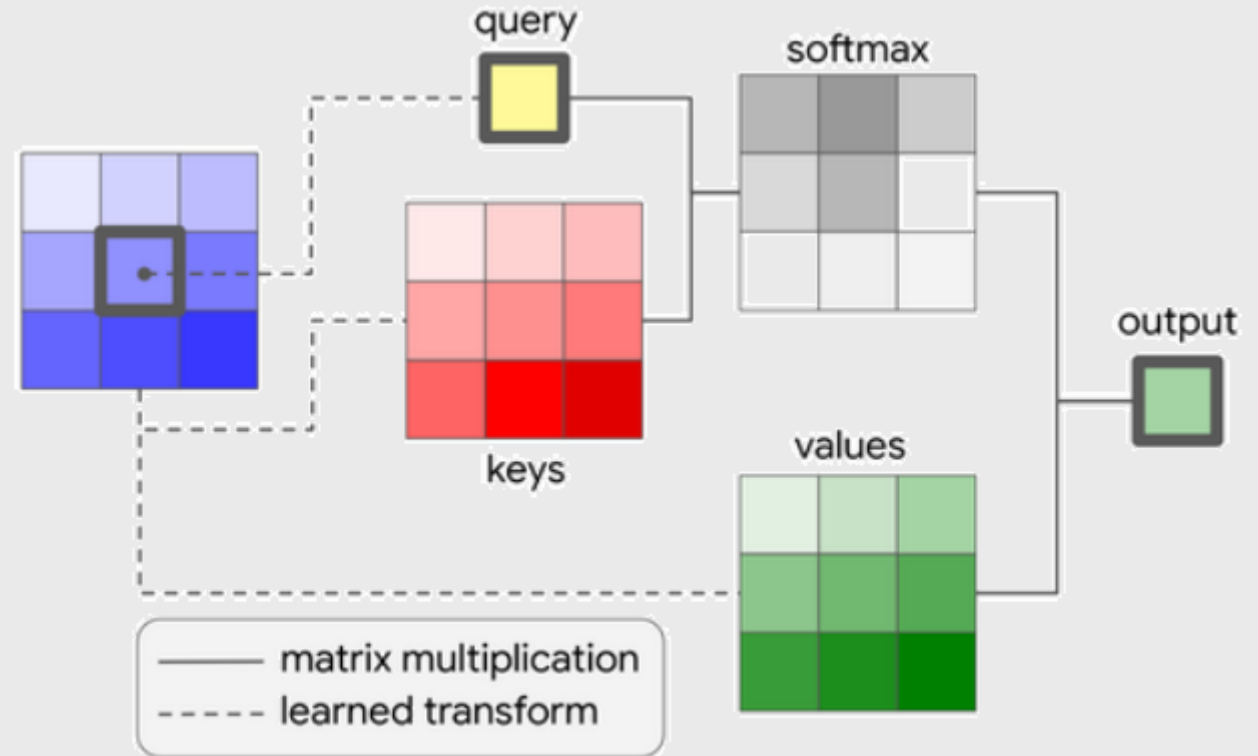
Convolutional Nets Vs Transformer

Ramachandran et al. Stand-alone self-attention in vision models. NIPS 2019



3×3 convolution.

The output is the inner product between the local window and the learned weights

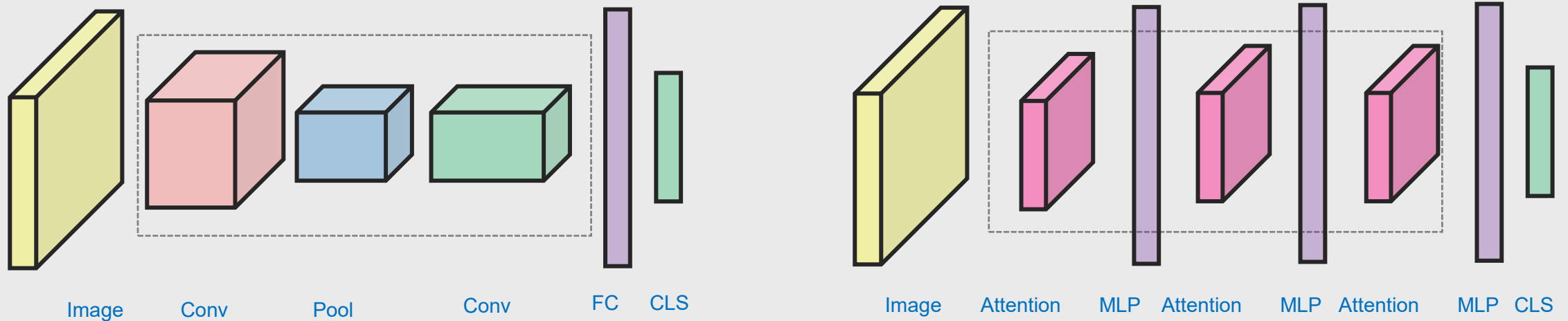


Self-attention around image local region

The output is local self attention/ on global receptive field

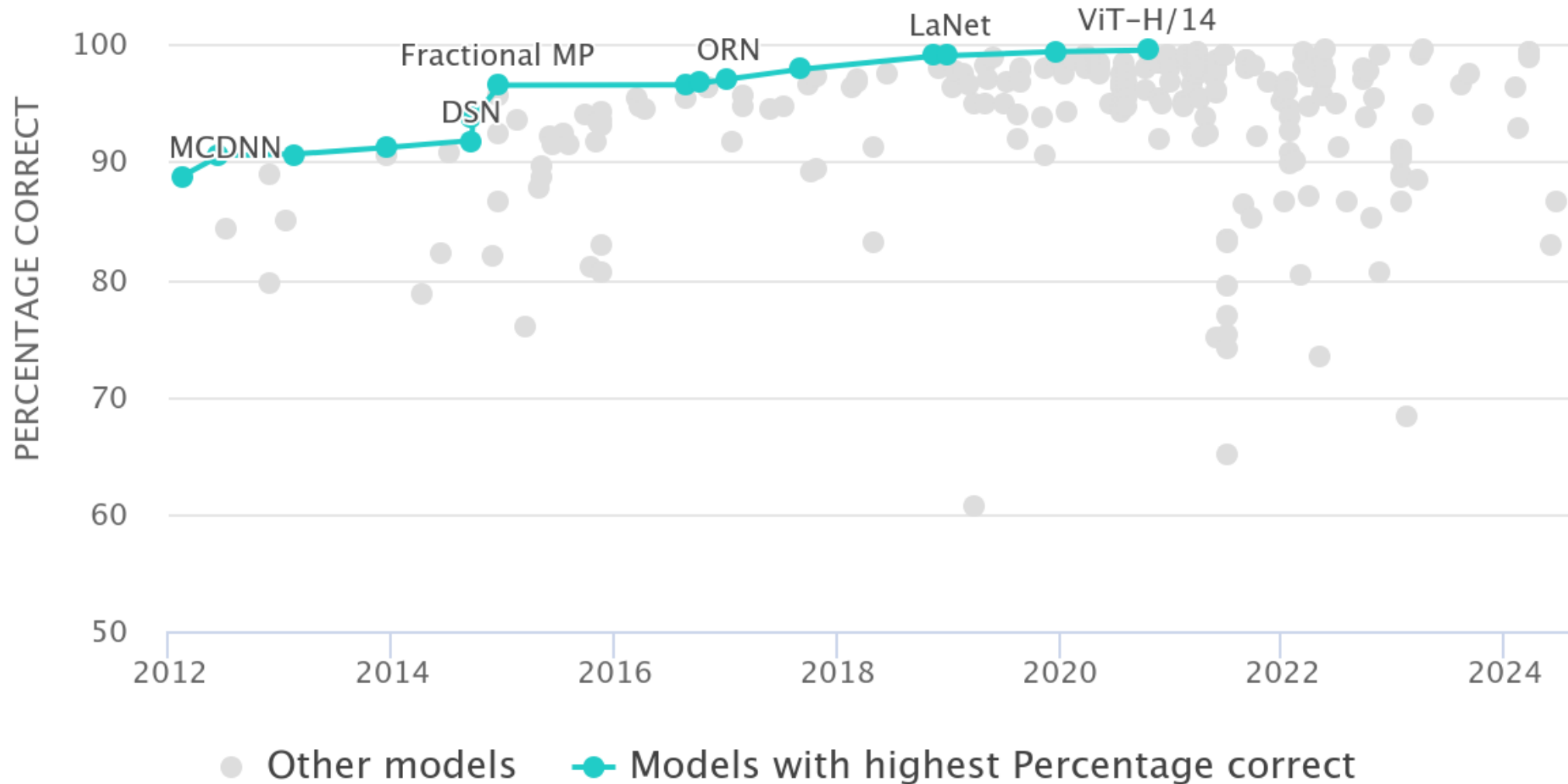
Standalone Self-attention in Vision Models

Ramachandran et al. Stand-alone self-attention in vision models. NIPS 2019



	ResNet-26			ResNet-38			ResNet-50		
	FLOPS (B)	Params (M)	Acc. (%)	FLOPS (B)	Params (M)	Acc. (%)	FLOPS (B)	Params (M)	Acc. (%)
Baseline	4.7	13.7	74.5	6.5	19.6	76.2	8.2	25.6	76.9
Conv-stem + Attention	4.5	10.3	75.8	5.7	14.1	77.1	7.0	18.0	77.4
Full Attention	4.7	10.3	74.8	6.0	14.1	76.9	7.2	18.0	77.6

Convolutional Nets Vs Transformers



AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

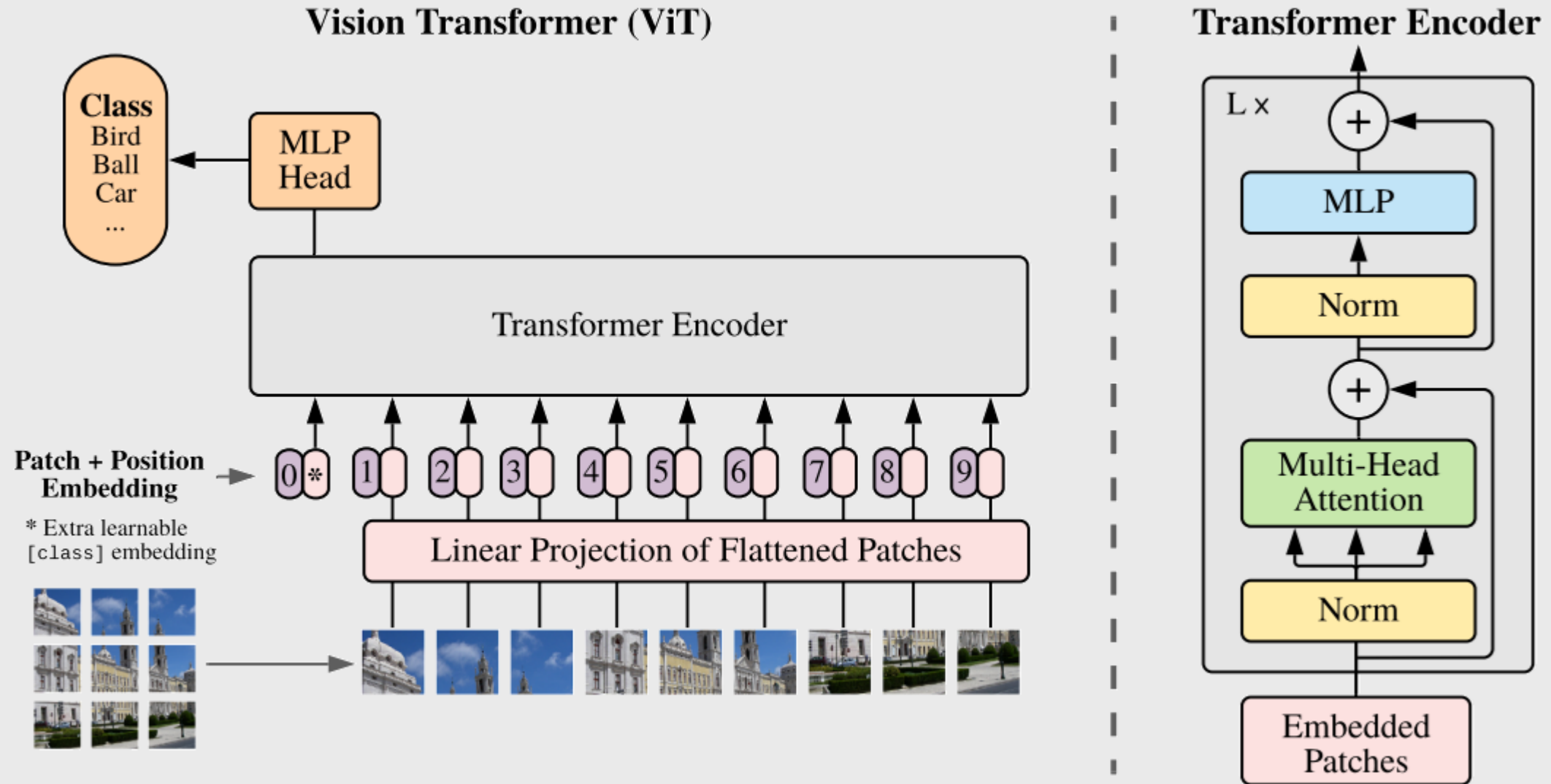
Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.¹

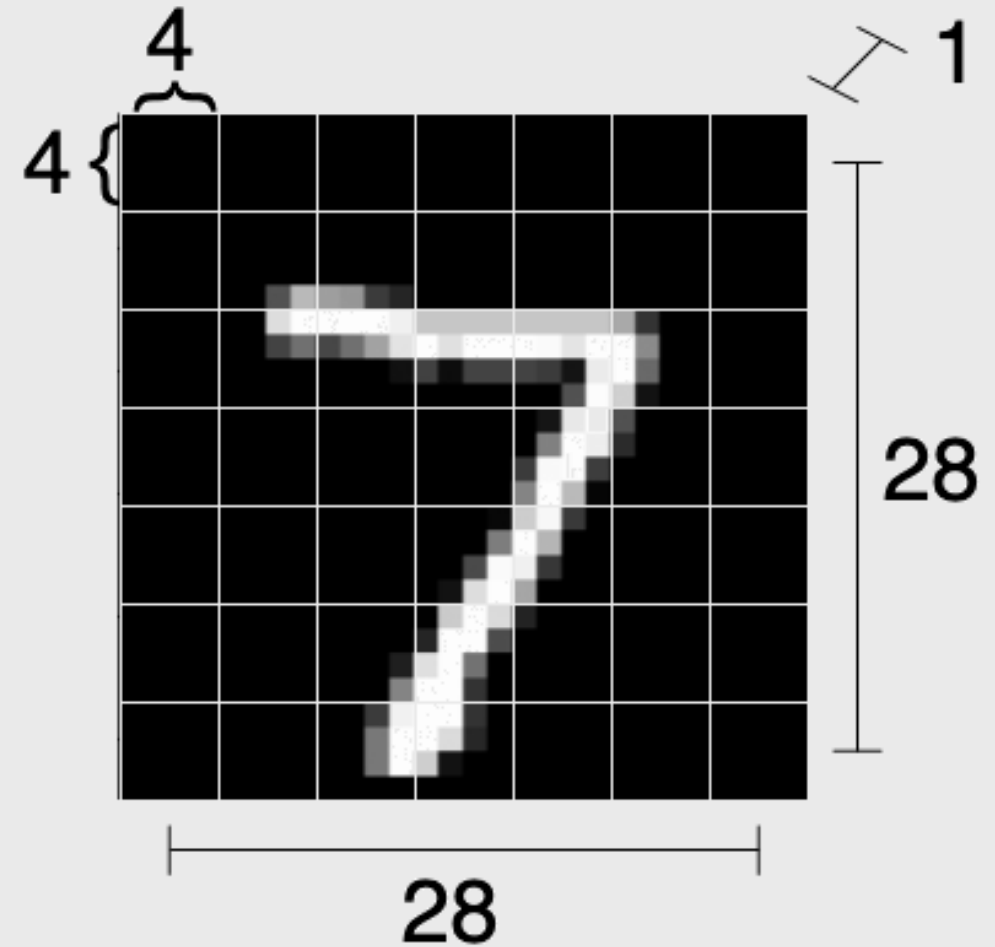
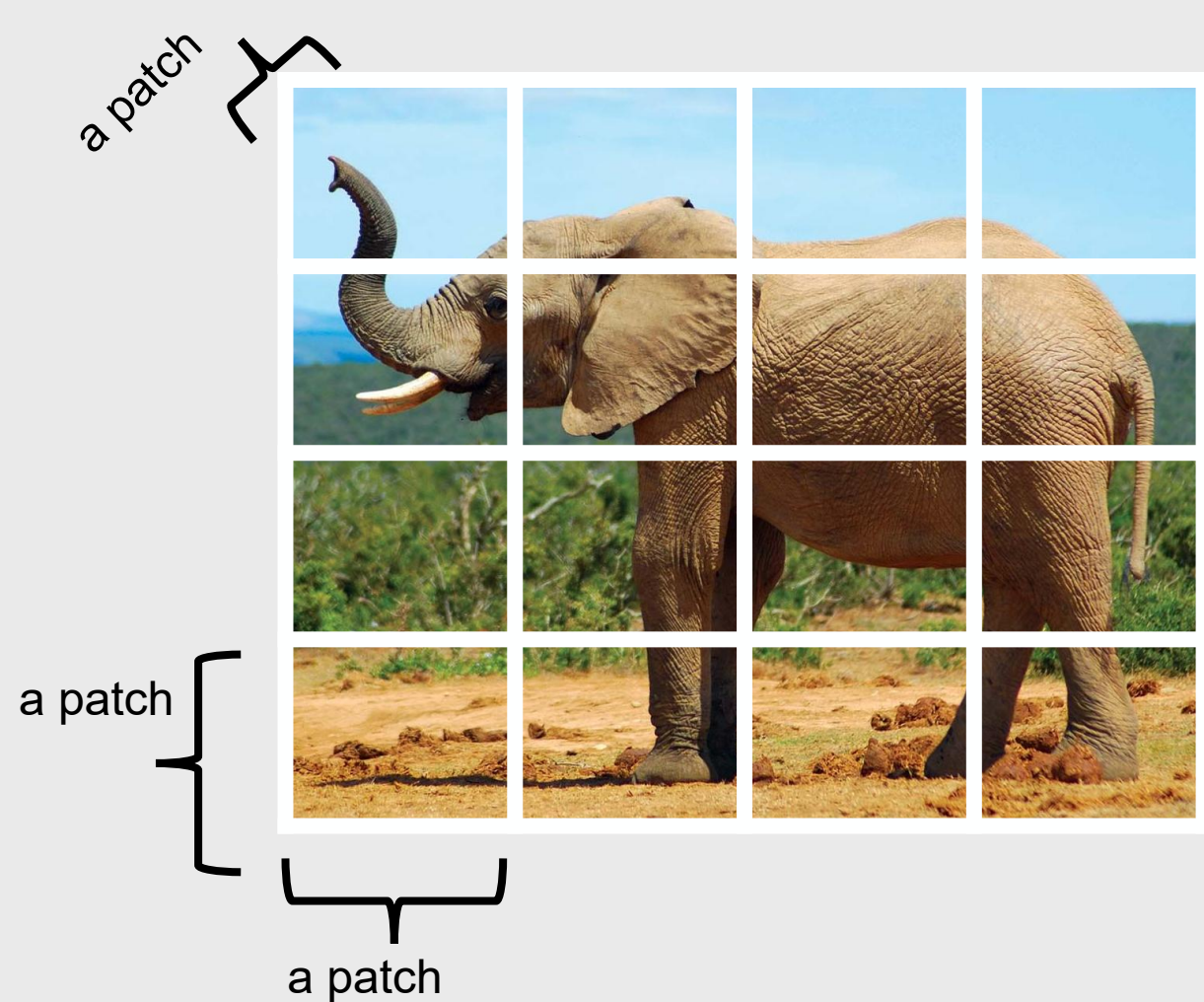
How Vision Transformer Models Works



Splitting an Image into Patches

6:31 PM

Split the image into patches, each of size (H'xW'xD)

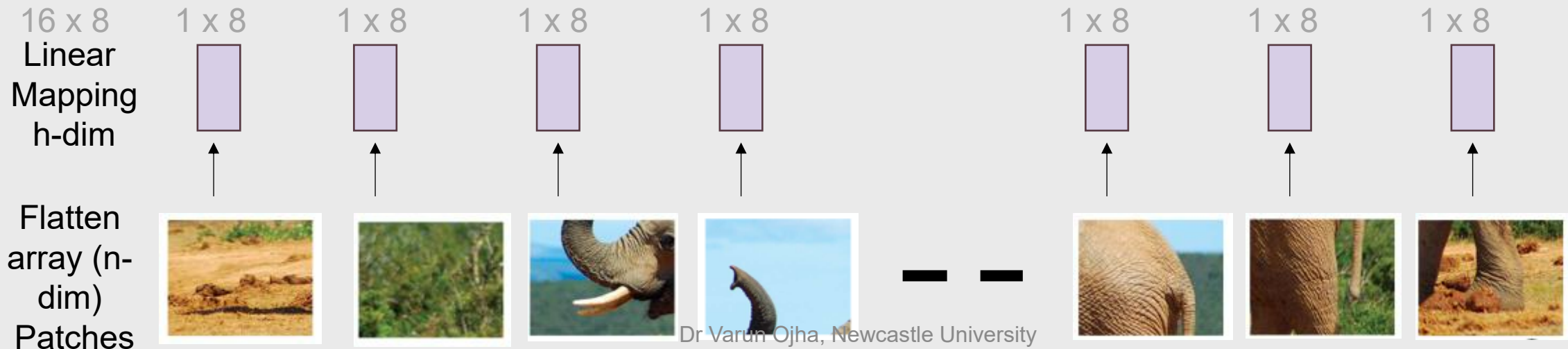
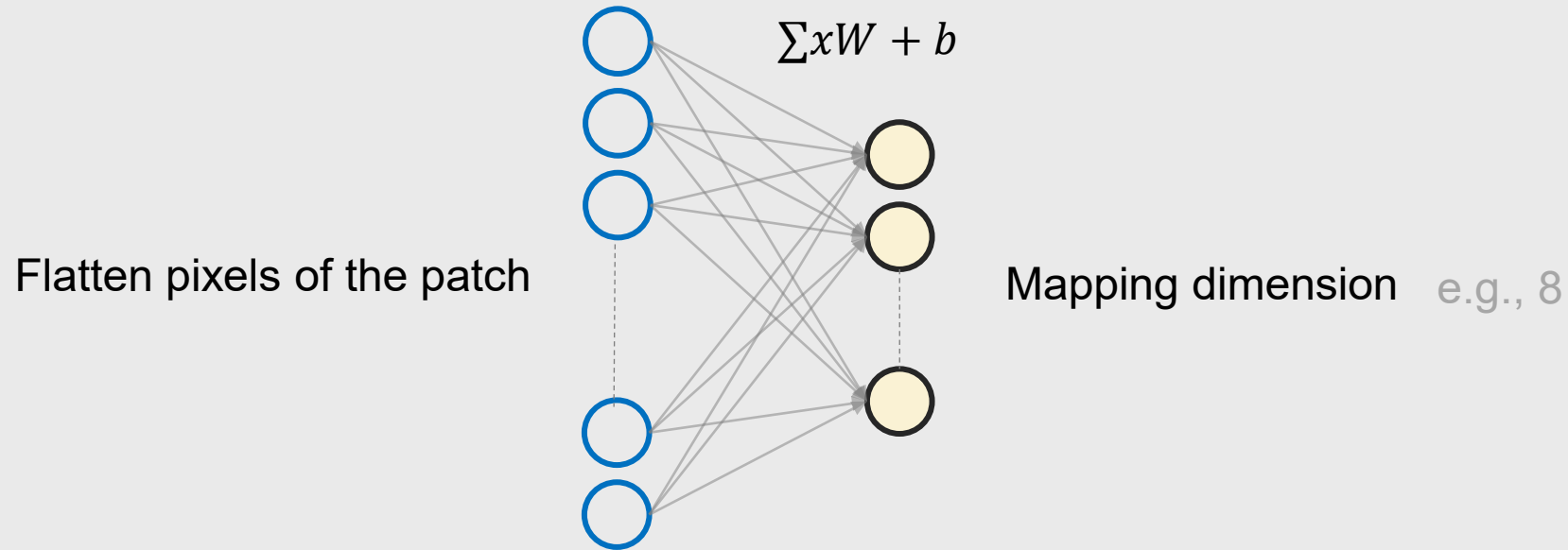


MIST dataset

Linear mapping: Embedding

6:31 PM

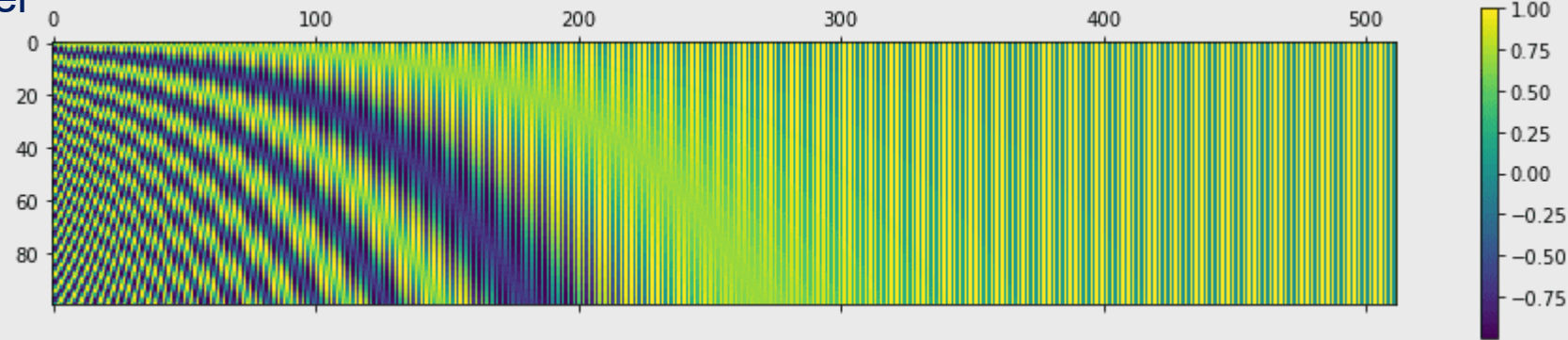
Linear projection to D-dimensional vector



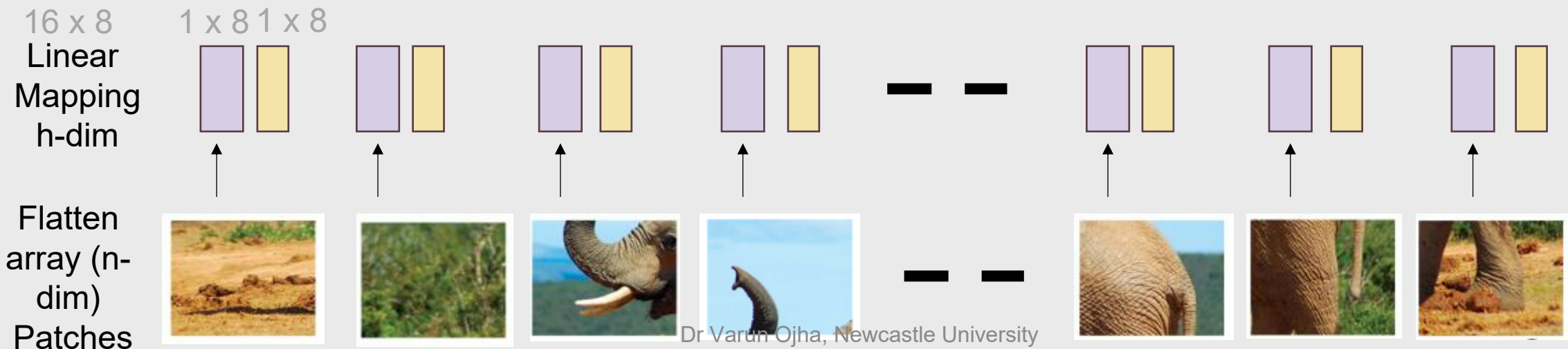
Positional Encoding

6:31 PM

Inform the model where the patch's position in the image is. In other word use sine and cosine values for respective patch number



$$p_{i,j} = \begin{cases} \sin \left(\frac{i}{10000^{\frac{j}{d_{emb-dim}}}} \right) & \text{if } j \text{ is even} \\ \cos \left(\frac{i}{10000^{\frac{j-1}{d_{emb-dim}}}} \right) & \text{if } j \text{ is odd} \end{cases}$$

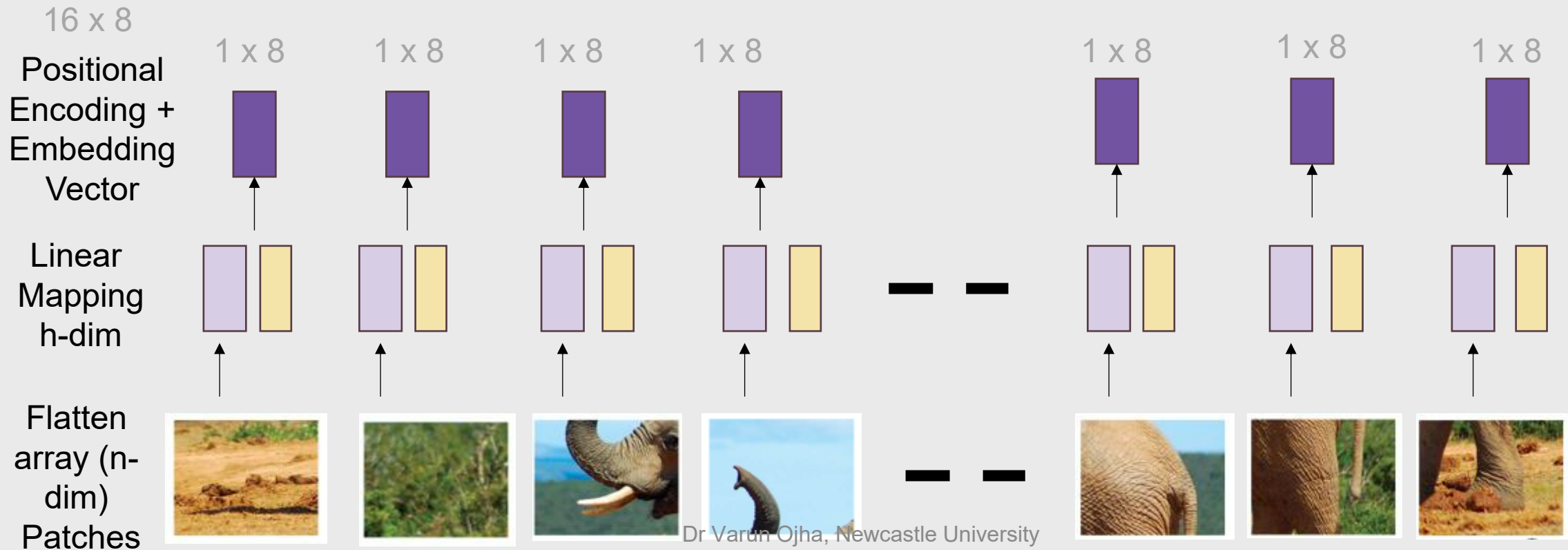


Positional Encoding and Embedding Vectors

6:31 PM

Inform the model where the patch's position in the image is. In other word use sine and cosine values for respective patch number

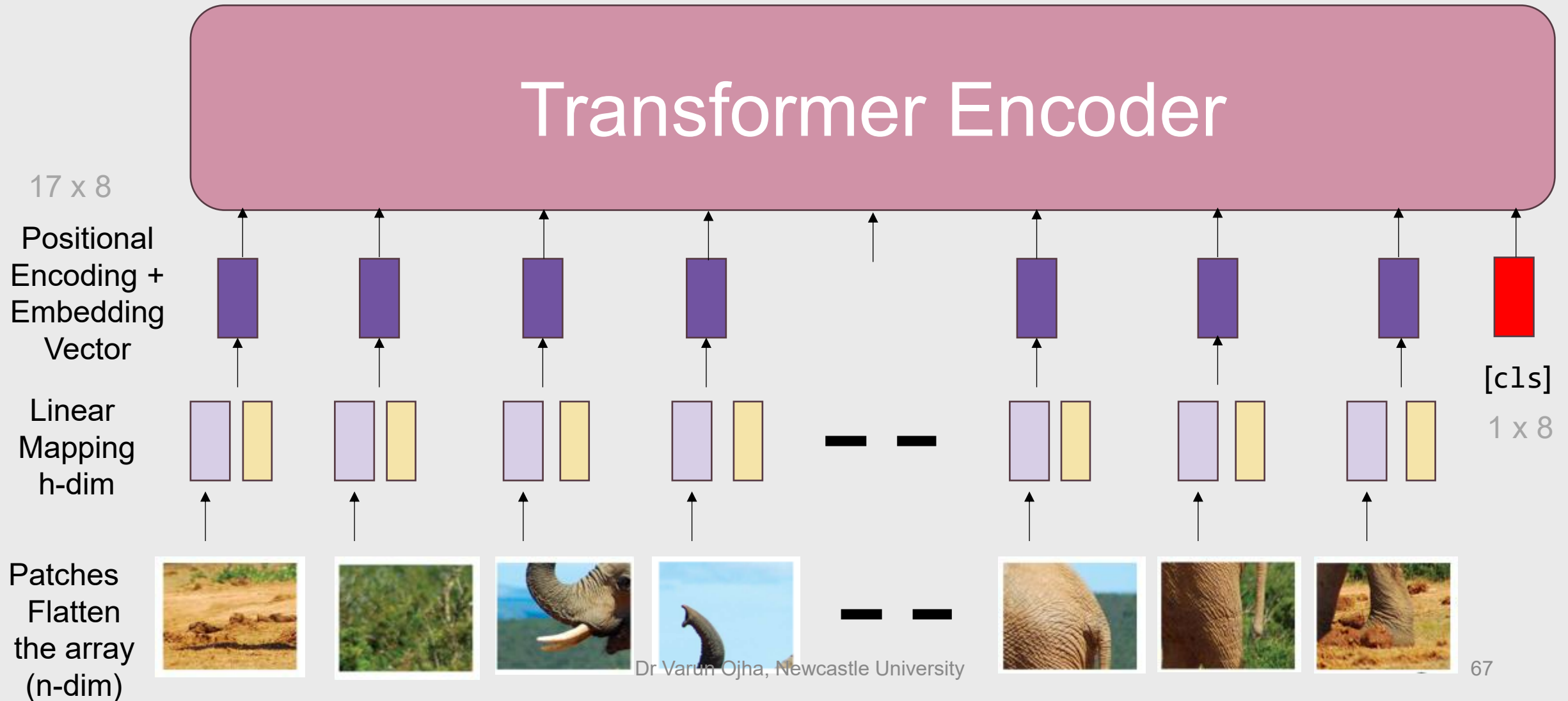
$$p_{i,j} = \begin{cases} \sin\left(\frac{i}{10000^{\frac{j}{d_{emb-dim}}}}\right) & \text{if } j \text{ is even} \\ \cos\left(\frac{i}{10000^{\frac{j-1}{d_{emb-dim}}}}\right) & \text{if } j \text{ is odd} \end{cases}$$



Add a Learnable Classification Token

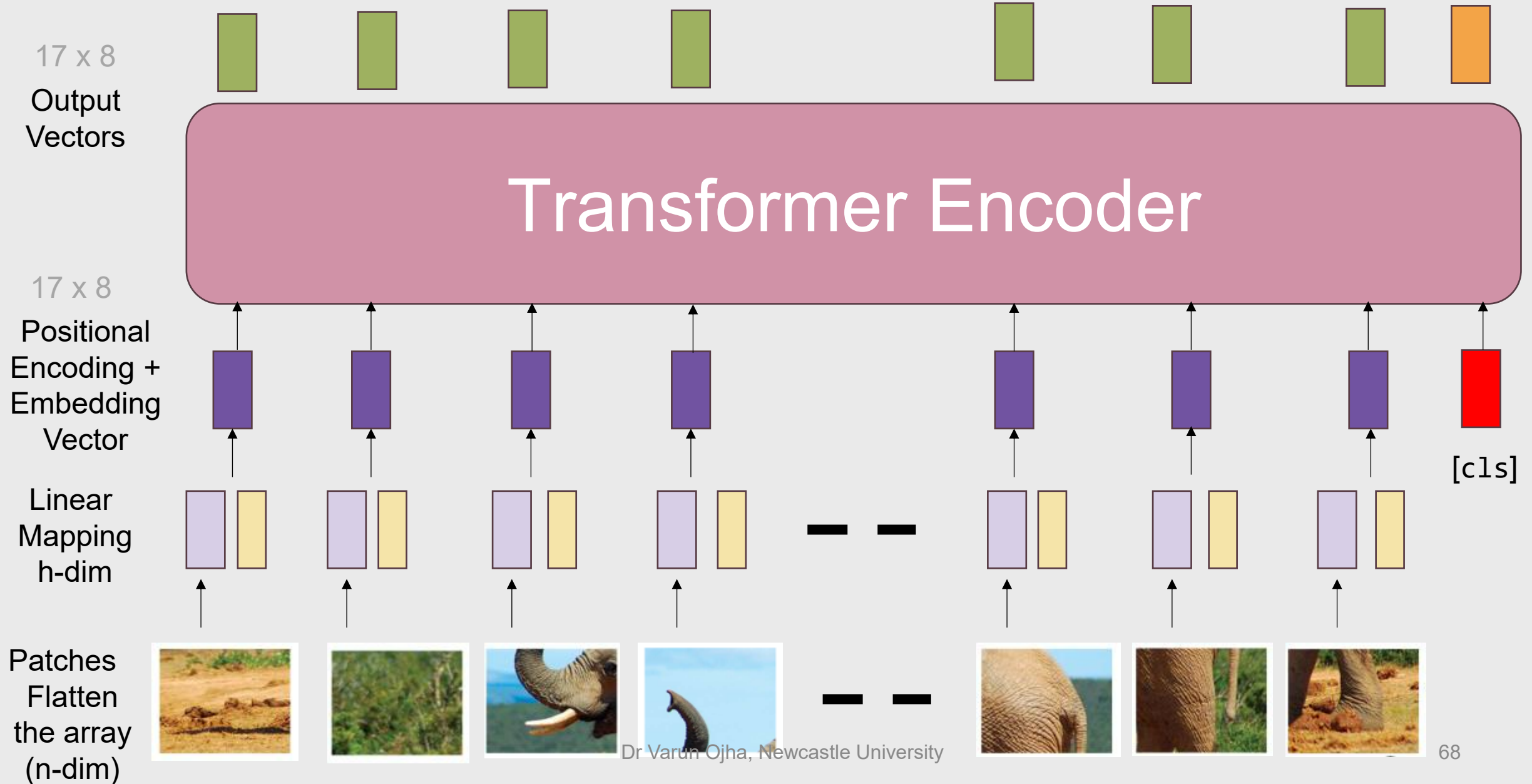
6:31 PM

A single, dedicated vector that serves as the **aggregated representation** for the entire input image, enabling image-level classification



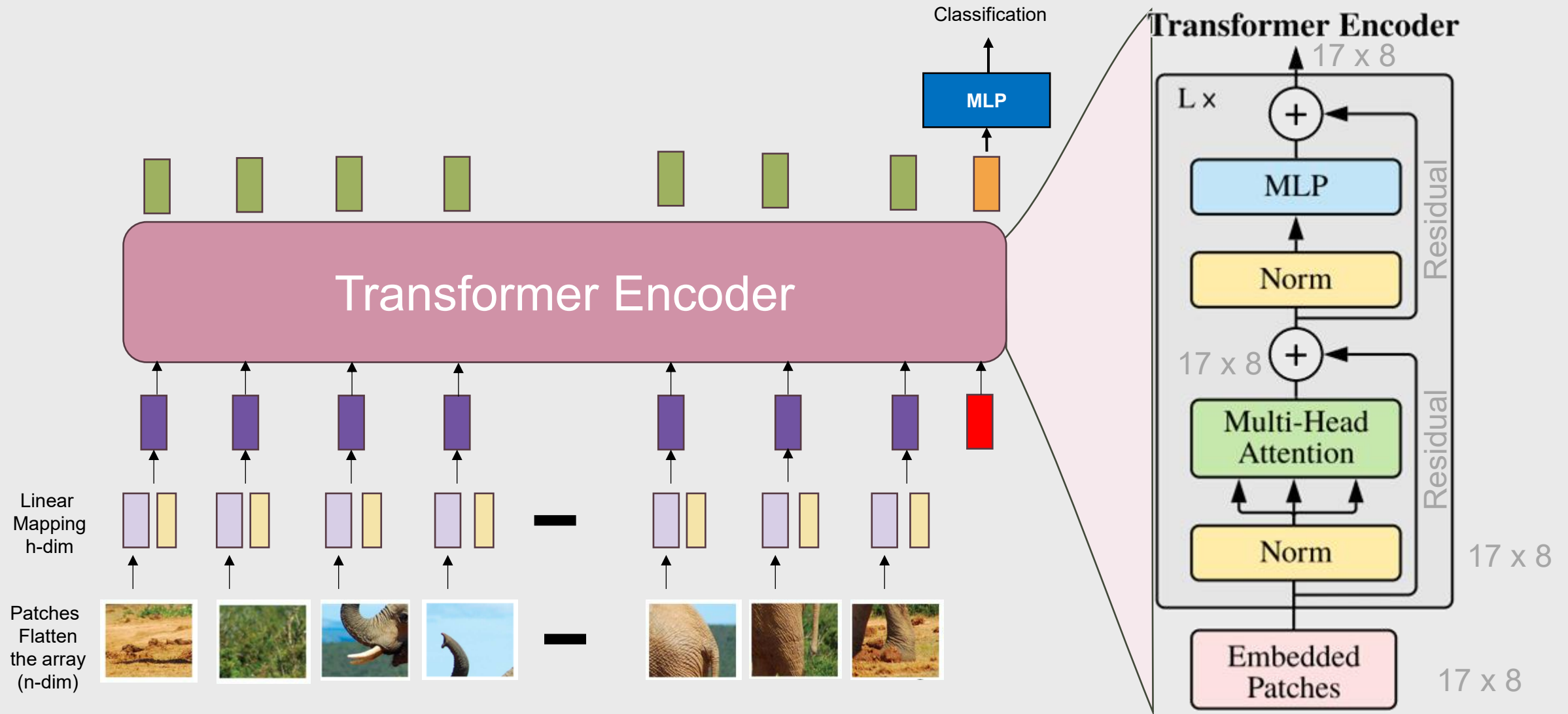
Output Vectors of Transformer Encoder

6:31 PM
1 x 8

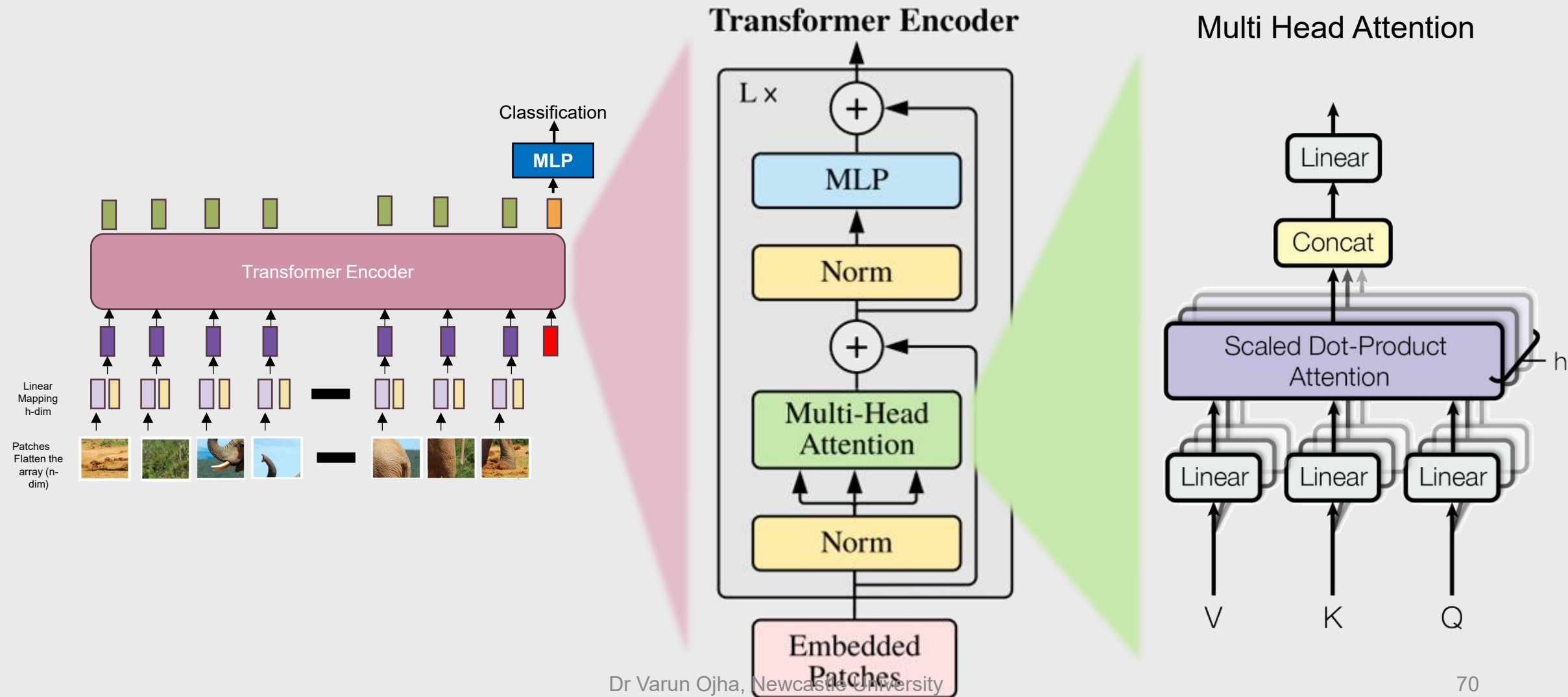


Same as Text Transformer

6:31 PM

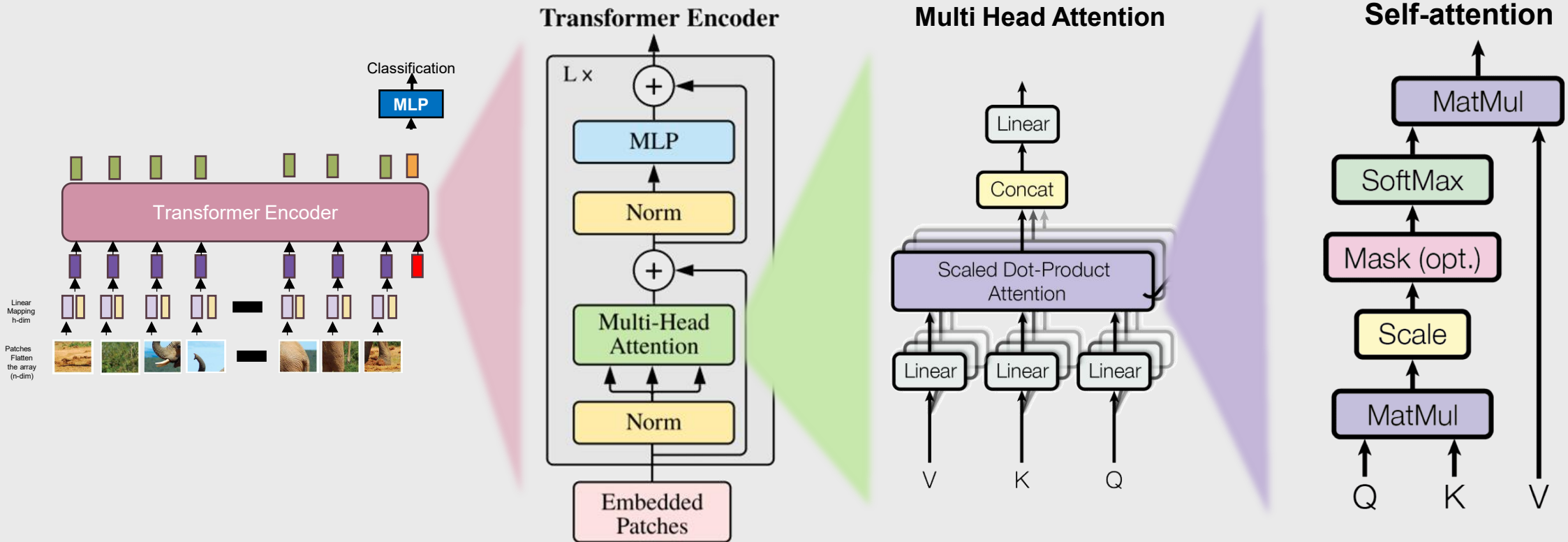


Same as Text Transformer



Same as Text Transformer

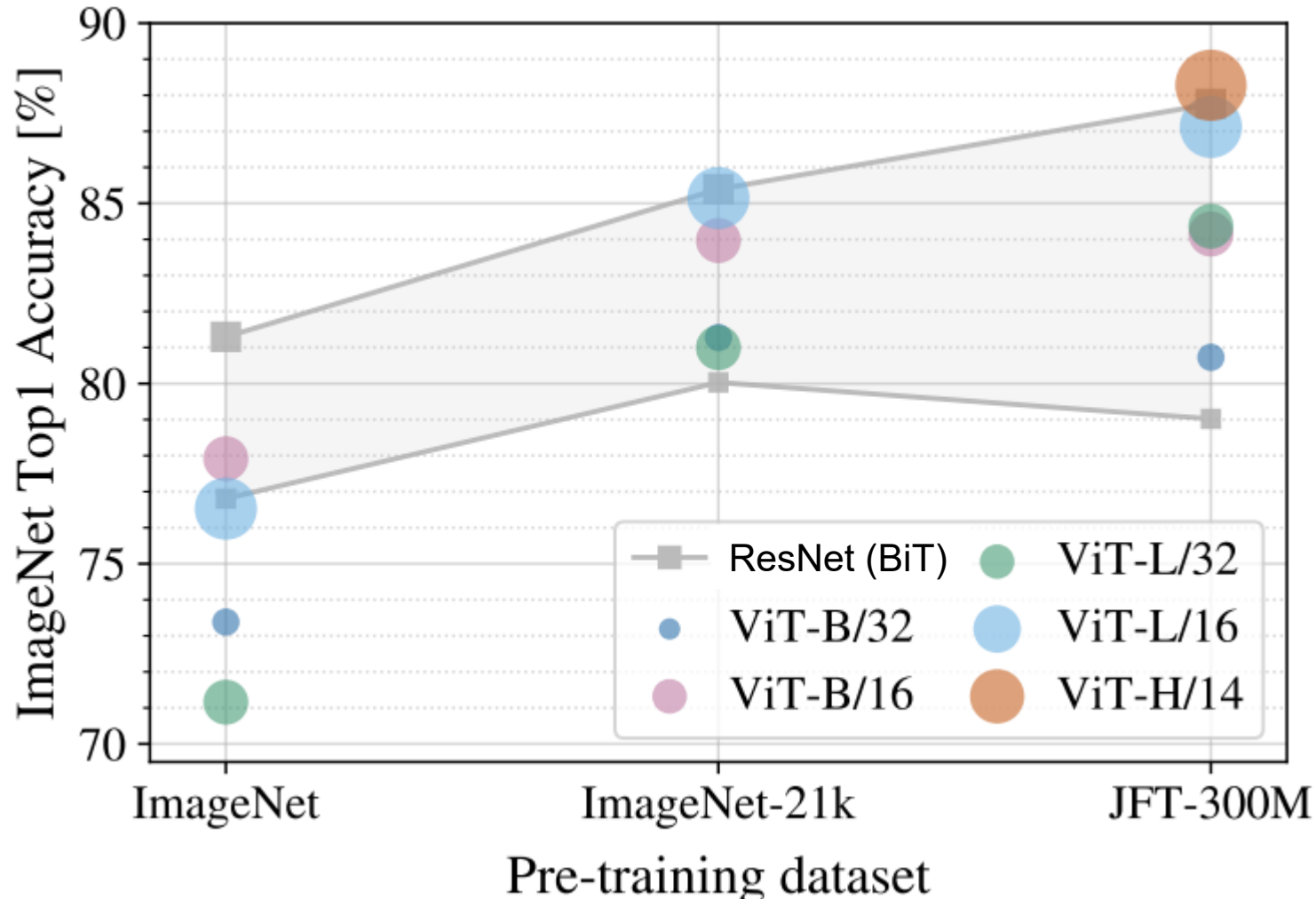
6:31 PM



$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

ViT Performance

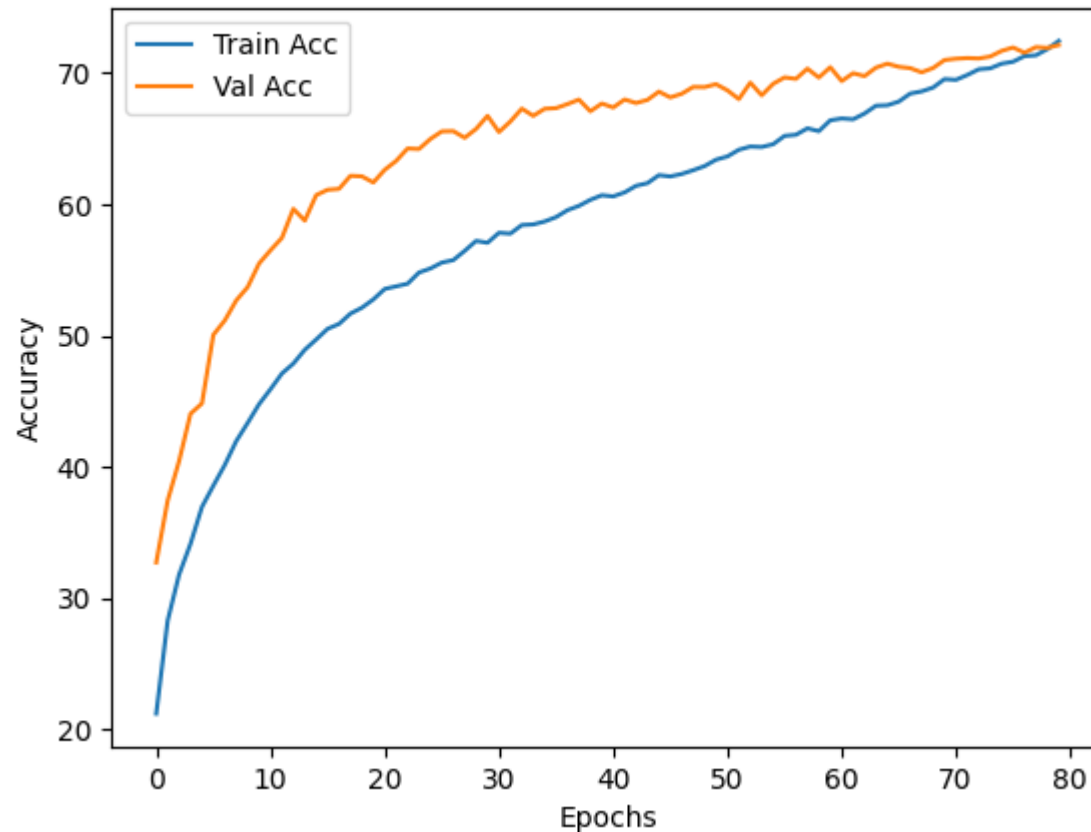
Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, ICLR 2017



Note that this performance is only achieved when ViT is pre-trained on large dataset (in this case JFT-300M is a 300 million image dataset of Google)

ViT on CIFAR-10 (without Pre-Training)

Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, ICLR 2017



Note that the performance depends on hyper parameter tuning models' size etc.

ViT Performance

- Worse than ResNet when trained just on ImageNet
- Performance improved when pre-trained on very large dataset
- Pretrained outperforms much bigger CNNs
- You need large GPUs (Computational Cost is very high)